

Rohan Jai Dharmaraj

EVALUATING TRANSFORMER AND LLM CONSTRAINTS FOR TRACEABILITY LINK RECOVERY

Faculty of Information Technology and Communication Sciences (ITC)

Master's thesis

Supervisor: Kari Systä

Supervisor: Zheyang Zhang

March 2026

Abstract

Rohan Jai Dharmaraj: EVALUATING TRANSFORMER AND LLM CONSTRAINTS FOR TRACEABILITY LINK RECOVERY

Master's thesis

Tampere University

Master's Degree Programme in Signal Processing and Machine Learning

March 2026

Modern software evolves rapidly, and the rationale behind changes often fades as issues are closed and commits accumulate. This makes it difficult for developers to determine which commit implemented a given issue or requirement, or which issue explains a particular code change. Requirements traceability addresses this problem by linking related development artifacts, but such links are rarely maintained consistently in real GitHub repositories. This thesis investigates automated Traceability Link Recovery (TLR) for issue–commit linking under realistic repository conditions, with a particular focus on improving precision and reducing the verification burden on developers. The work follows an iterative empirical design in which each experimental step was motivated by limitations observed in the previous one. First, the study revisits the T-BERT baseline by restructuring commit diffs to preserve edit order and change direction, following a future-work direction proposed in prior research. The goal was to test whether a more structured input representation could increase the precision of the model. The results show that these ordered diff variants do not improve performance and slightly underperform the original baseline. Second, after these limitations were attributed in part to the representational and context constraints of the T-BERT architecture, the study evaluates a Small Language Model (SLM) in a zero-shot setting that directly decides whether an issue and a commit are linked. This approach achieves very high recall, but also produces many false positives, resulting in poor precision, especially in larger repositories. Third, Retrieval-Augmented Generation (RAG) is introduced as a repository-grounding mechanism that retrieves semantically related commit fragments as evidence for each issue and injects them into the SLM prompt, while also reducing exhaustive pairwise prompting. Retrieval depth K is treated as a design factor, larger K generally recovers more true links but also increases the number of predicted links, revealing a clear trade-off between recall and verification effort. Overall, the thesis shows that practical traceability link recovery depends not only on model choice, but also on input representation, context limits, calibration, and retrieval design. More reliable automation will require tighter retrieval, stronger filtering, and task-specific adaptation beyond prompt-based inference alone.

Keywords: traceability, issue-commit links, transformer models, small language model, retrieval-augmented generation, software engineering, context-window.

The originality of this thesis has been checked using the Turnitin Originality Check service.

Use of AI in Thesis

I confirm that AI-based tools were used in the preparation of this thesis:

☐ No

☒ Yes

The AI-based applications used in the development of this work are:

- ChatGPT
- Perplexity
- Grammarly

Purpose of the Use of AI

Tools like ChatGPT, Perplexity and Grammarly were used for brainstorming, paraphrasing, improving clarity of expression, structuring sections, verifying technical explanations, resolving LaTeX-related issues, and conducting preliminary literature searches during the background research phase.

Parts of this work, where AI was used

AI tools were used in the following parts of this thesis:

- Chapter 1: Introduction – language refinement and clarity improvement
- Chapter 2: Related Work – Literature searches, summarization guidance, and paraphrasing assistance
- Chapter 3: Research Methodology – structural suggestions, technical description verification, diagram conceptualization and LaTeX syntax assistance
- Chapters 4 and 5: Results and Discussion – structural recommendations, paraphrasing support, and assistance with technical implementation, LaTeX assistance
- Chapter 6: Conclusion – language refinement and sentence clarity improvement

Preface

The completion of this stage of my academic journey marks an important milestone in my life, and I would like to take this opportunity to acknowledge the individuals who have supported and guided me along the way.

First and foremost, I extend my deepest gratitude to my supervisor, Kari Systä, for giving me the opportunity to be part of his research group and for his continuous guidance and mentorship. His expertise, patience, and constructive feedback have been invaluable. Beyond academic guidance, his encouragement and thoughtful advice have helped me grow in confidence, independence, and critical thinking.

I am also deeply thankful to Zheyang Zhang for her consistent support and willingness to help whenever challenges arose. Whether discussing research ideas, offering insightful suggestions, or providing practical advice during demanding periods, her guidance made a meaningful difference. Her support has contributed greatly to my development during this time.

My sincere thanks also go to the entire research team for fostering a collaborative and supportive environment. The discussions, shared experiences, and mutual encouragement within the group enriched my understanding and inspired me to approach challenges from different perspectives. Being part of such a team has been both motivating and rewarding.

Finally, I am profoundly grateful to my parents and friends for their unwavering love, encouragement, and belief in me. Their constant support, patience, and understanding have provided the strength needed to overcome challenges and remain committed to my goals. This achievement would not have been possible without them.

Tampere, March 2026

Rohan Jai Dharmaraj

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Problem Statement and Research Gaps	3
1.3	Research Objectives	4
1.4	Thesis Contributions	5
1.5	Thesis Structure	6
2	Related Work	7
2.1	Requirements Traceability and Traceability Link Recovery	7
2.2	Information Retrieval Approaches to Traceability	7
2.3	Learning-Based Approaches	8
2.3.1	Deep Learning Models	9
2.4	Transformer-Based Traceability Models	9
2.4.1	T-BERT and Its Practical Input Limitations	10
2.4.2	Other Transformer-Based Models	11
2.5	Small Language Models for Traceability	12
2.5.1	Rationale for Selecting Code Llama 7B Instruct	13
2.6	Retrieval-Augmented Generation and Repository Context	13
2.7	Summary and Research Gaps	14
3	Research Methodology	15
3.1	Input Artifacts Definition	15
3.1.1	Issue Artifact	16
3.1.2	Commit Artifact	16
3.1.3	Commit Diff Normalization Across Model Families	17
3.2	RQ1: T-BERT Commit Diff Restructuring Experiment	18
3.2.1	Effect of Commit Diff Ordering and Input Construction	18
3.2.2	Input Configurations for T-BERT	21
3.2.3	Training and Cross-Project Evaluation Datasets	22
3.3	RQ2: Zero-Shot Traceability With a SLM	22
3.3.1	Model Selection and Zero-Shot Setting	22
3.3.2	Model Configuration	23
3.3.3	Data Acquisition and Preprocessing	23
3.3.4	Execution Protocol	23
3.4	RQ3: Repository-Grounded Traceability via RAG	24
3.4.1	Repository Context Preparation and Indexing	25
3.4.2	Evaluation Repositories	25

3.4.3	Retrieval and Prompt Construction	26
3.4.4	Retrieval Depth as an Experimental Factor	27
3.5	Evaluation Protocol and Metrics	27
3.5.1	Mapping Reference Labels	27
3.5.2	Classification Metrics: Precision and Recall	28
3.5.3	Ranking Metrics: Precision@K, MAP@3, and MRR	28
3.5.4	Metric Usage Across RQ1–RQ3	29
4	Results	31
4.1	RQ1: Impact of Commit Input Restructuring on T-BERT Performance	31
4.1.1	Cross-Project Results on Benchmark Repositories	31
4.2	RQ2: Traceability Performance of a SLM Compared to T-BERT . . .	33
4.2.1	Results Across Evaluation Projects	33
4.3	RQ3: Impact of RAG-Based Repository Grounding on Traceability .	34
4.3.1	Effect of Retrieval Depth (K) on Link Recovery	34
4.3.2	Prediction Volume and Precision Trade-Off	35
4.3.3	Comparison with T-BERT Baseline	36
5	Discussion	37
5.1	RQ1: Impact of Input Restructuring on T-BERT Performance	37
5.2	RQ2: The SLM Behavior and the Over-Linking Problem	39
5.3	RQ3: RAG Improves Recall but Increases Verification Effort	41
5.4	Recommendations and Future Work for Building Traceability Tools .	42
5.5	Threats to Validity	43
6	Conclusion	45
	References	53
	APPENDIX A. Prompt Templates	54

List of Figures

2.1	Multi-phase training strategy of T-BERT, consisting of code-aware initialization, intermediate alignment training, and traceability-specific fine-tuning	10
3.1	Example of artifact construction used to build Issue and Commit artifacts	17
3.2	Example of a commit diff from flask repository	19
3.3	Fine-tuning setup for T-BERT using two commit artifact variants (V1 and V2)	21
3.4	Zero-shot prompting workflow for the generative SLM	24
3.5	RAG pipeline for repository-grounded traceability decisions	27
4.1	RAG performance as a function of retrieval depth (K) across all evaluation projects, showing total predictions (top-left), true positives recovered (top-right), and false positives (bottom-left).	34
5.1	Token Length Distribution by Project for Baseline T-BERT Inputs	38
5.2	Token Length Distribution by Project for Version 1 and Version 2 Inputs	39
5.3	Proportion of SLM prompt-2 that exceed the model context window and are truncated, reported per project	41

List of Tables

3.1	Repositories used for T-BERT fine-tuning and cross-project evaluation	22
3.2	Generation parameters used for deterministic zero-shot inference with the SLM	23
3.3	Summary of evaluation repositories used for SLM and RAG experiments (RQ2 and RQ3)	26
4.1	T-BERT Ranking Performance with Confusion Matrix Statistics . . .	31
4.2	Zero-shot SLM vs. T-BERT with Confusion Matrix Statistics	33
4.3	RAG results across retrieval depths K shown as TP/(total predictions), with Precision and Recall at each K	35

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
COT	Chain Of Thought
GRU	Gated Recurrent Unit
IR	Information Retrieval
LLM	Large Language Model
LSI	Latent Semantic Indexing
LSTM	Long Short Term Memory
MAP	Mean Average Precision
ML	Machine Learning
MRR	Mean Reciprocal Rank
NLP	Natural Language Processing
ONS	Online Negative Sampling
OSS	Open Source Software
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RNN	Recurrent Neural Network
SLM	Small Language Model
TF-IDF	Term Frequency - Inverse Document Frequency
T-BERT	Traceability - Bidirectional Encoder Representations from Transformers
TLR	Traceability Link Recovery
VSM	Vector Space Model

1 Introduction

1.1 Background and Motivation

Modern software systems evolve continuously through bug fixes, feature additions, and refactoring [41]. As systems grow in size and complexity, understanding why a particular change was made and which requirement it addresses becomes increasingly difficult. Requirements traceability aims to address this challenge by establishing explicit links between requirements and downstream artifacts such as issues, commits, and tests [7]. These links support key software engineering activities, including impact analysis, maintenance, compliance verification, and knowledge transfer within development teams.

Despite its importance, requirements traceability is often poorly maintained in practice. Creating and updating traceability links is largely a manual task that depends on consistent developer effort. In real-world projects, especially those developed on collaborative platforms such as GitHub [13], traceability frequently emerges as a by-product of developer habits rather than as a systematically managed process. As a result, links between issues and commits are often missing, inconsistent, or outdated, limiting their usefulness during later stages of maintenance and evolution. To address this gap, researchers have increasingly explored Traceability Link Recovery (TLR), which focuses on automatically identifying missing links between software artifacts after development has taken place. TLR methods aim to reduce manual effort while improving the completeness and reliability of traceability information in real-world repositories.

Early approaches to traceability link recovery relied primarily on information retrieval and heuristic techniques that compare textual similarity between artifacts such as requirements documents, issues, and source code [2, 26]. Candidate links were typically proposed based on keyword matching, vector-space similarity, or repository conventions such as issue identifiers embedded in commit messages. While these methods provided a degree of automation, their effectiveness was limited when artifacts used different terminology, lacked detailed documentation, or exhibited inconsistent writing practices.

As research in traceability link recovery progressed, learning-based approaches were increasingly adopted to better capture semantic relationships between natural-language artifacts and technical artifacts. In particular, transformer-based models have shown strong performance in this domain. One influential work is T-BERT [23], which adapts a transformer-based language model to recover links between GitHub issues and commits. T-BERT demonstrated that learning-based models can out-

perform traditional similarity-based techniques when trained on suitable datasets [27], and it has since become a widely used baseline in traceability research. Building on this direction, subsequent models such as BTLINK [21], DSSLINK [47], and MTLINK [9] further explore pre-trained representations, semi-supervised learning, and multi-task formulations to improve link recovery in challenging settings. More recently, research has also begun to investigate the use of Large Language Models and retrieval-augmented pipelines for traceability-related tasks, where external context is retrieved and provided to the model to support link generation or ranking [17].

However, most evaluations of T-BERT have been conducted under controlled experimental conditions, using curated datasets with relatively consistent documentation practices. This raises important questions about how well such models perform in realistic software repositories, where issues may be vague, commit messages incomplete, and development activity highly uneven.

The present thesis builds directly on prior research in automated traceability link recovery, especially transformer-based models for issue–commit linking in real-world repositories, as well as on a recent Master’s thesis by Parveen [30], which systematically evaluated T-BERT in real-world GitHub projects. That study demonstrated that while T-BERT is capable of recovering meaningful links, its reliability decreases in repositories characterized by noisy data, limited documentation, and inconsistent linking practices. The findings highlighted several practical challenges, including overconfident predictions for weakly described artifacts and a lack of project-specific contextual awareness.

While prior work has largely focused on improving performance under curated benchmark conditions or proposing new model architectures, this thesis shifts the focus toward addressing the practical limitations observed when applying traceability models in real repositories, as shown by Parveen [30]. Rather than introducing a new architecture, the thesis first revisits the widely used T-BERT baseline by examining alternative input restructuring strategies that preserve the sequential nature of code changes. At the same time, the limitations observed in encoder-based models motivate the exploration of Small Language Models as an alternative approach, since they allow the task to be framed as an explicit decision about whether an issue and a commit are linked and may better leverage instruction following and reasoning over heterogeneous artifacts. In addition, the thesis investigates the use of Retrieval-Augmented Generation to inject repository-level context, motivated by the observation that accurate linking often depends on project-specific evidence beyond the isolated issue–commit pair [30]. By addressing the limitations identified in real-world evaluations, this thesis aims to better understand how automated traceability can be made more robust, reliable, and practically useful for software

engineers.

1.2 Problem Statement and Research Gaps

Based on prior work and real-world observations, several open challenges remain in automated traceability link recovery:

1. T-BERT and similar issue–commit link recovery pipelines (including later variants such as BTLINK, DSSLINK, and MTLINK) typically require aggressive preprocessing of commit diffs to fit model input limits [23, 21, 47, 9]. In T-BERT specifically, commit diffs interleave logically related edits and remove cues about how a change unfolds across files or hunks, potentially weakening the representation of implementation context [23, 30]. This motivates investigating whether preserving ordering and change direction within the same T-BERT pipeline improves link recovery behavior in realistic repositories.
2. Most existing approaches rely on encoder-only models such as T-BERT and its architectural variants [23, 9, 21, 47], which are trained specifically for similarity prediction. This leaves open the question of how decoder-only models, particularly Small Language Models (SLMs), perform when adapted to the traceability task. SLMs enable an alternative formulation in which the model is instructed to *decide* whether an issue and a commit are linked, rather than merely producing similarity scores. However, it remains unclear whether the decisions produced by SLMs remain reliable in the presence of noisy issue descriptions and large code diffs, and how such approaches compare to the widely used T-BERT baseline.
3. Current models typically operate on isolated issue–commit pairs [23, 9, 21, 47] and do not sufficiently incorporate broader repository context or project history, which can reduce reliability and developer trust. In practice, developers often rely on the surrounding context when validating whether a commit truly addresses an issue. Retrieval-Augmented Generation offers a mechanism to inject such repository context at inference time [17], but how much repository grounding improves issue–commit traceability in realistic repositories remains underexplored. In addition, key retrieval design choices (such as retrieval depth K) may affect the trade-off between recovering true links and introducing noisy evidence.

Addressing these challenges is necessary to move automated traceability from a research prototype toward a tool that can be realistically integrated into software development workflows.

1.3 Research Objectives

The primary objective of this thesis is to improve the *precision* of automated traceability recovery in real-world GitHub repositories, building on the empirical findings reported by Parveen [30]. In that work, T-BERT recovered meaningful links but exhibited reduced reliability in noisy repositories, with a tendency to produce over-confident or weakly supported links. From a practical perspective, such behavior increases the verification burden on developers and reduces trust in automated traceability, making precision a critical target for improvement.

Following the future-work direction outlined in [30], this thesis first investigates whether improving the *input representation* to T-BERT can reduce false positives. The initial hypothesis was that preserving the sequential structure of code changes (i.e., ordering and change direction) would provide a stronger implementation context and help T-BERT discriminate true links from superficially similar pairs. When this strategy did not produce the expected precision gains, the study broadened to alternative approaches: (i) reformulating the task using Small Language Models that make an explicit linking decision, and (ii) grounding such decisions with repository-local evidence using Retrieval-Augmented Generation (RAG). These three approaches are captured by the following research questions:

RQ1: How do alternative input restructuring strategies affect traceability link recovery performance in T-BERT?

This question evaluates the hypothesis that order and direction-aware diff restructuring improves the quality of T-BERT predictions without changing the underlying architecture. Under fixed context limits, however, adding structure may reduce token coverage of the change, making the net effect uncertain. RQ1 therefore tests whether restructuring yields measurable precision improvements under the same training and evaluation protocol used for the baseline.

RQ2: How do Small Language Models perform on the traceability link recovery task compared to the T-BERT baseline?

After restructuring did not provide the expected precision gains, and given that the observed limitations were attributed in part to architectural constraints inherent to encoder-only models, RQ2 explores whether a different task formulation can produce more reliable link predictions. Specifically, the transition to Small Language Models is motivated by their ability to compare an issue and a commit more directly and decide whether they are linked. This question therefore, evaluates whether such a decision-oriented formulation improves precision relative to T-BERT.

RQ3: How does Retrieval-Augmented Generation (RAG) based repository grounding affect SLM-based traceability link recovery in real-world GitHub repositories?

Building on RQ2, RQ3 is motivated by an over-linking tendency observed in the SLM: while recall remained high, precision dropped sharply in larger repositories due to an increased number of false positives. RQ3 therefore, examines whether grounding SLM decisions with repository-local evidence can improve reliability and reduce spurious links. In the RAG setup, evidence is supplied by retrieving semantically related commit fragments from the same repository and injecting them into the prompt, which also avoids exhaustive pairwise prompting.

RQ3.1: How does retrieval depth (K) influence the precision–recall and verification–burden trade-off in the RAG-grounded setup?

Retrieving too little evidence may omit project-specific context that is necessary to validate a true link, while retrieving too much may introduce irrelevant material that encourages false positives. RQ3.1 therefore evaluates how varying K changes link recovery behavior and developer verification effort, and contextualizes these outcomes against the T-BERT baseline.

1.4 Thesis Contributions

This thesis makes the following contributions:

- An empirical evaluation of order and direction-aware commit diff restructuring within the standard T-BERT issue–commit linking pipeline, showing that these representation changes do not improve performance under fixed context-length constraints.
- A systematic characterization of zero-shot Small Language Model behavior for issue–commit traceability, showing a recall-oriented decision pattern with substantial precision loss due to over-linking as repository size increases.
- A Retrieval-Augmented Generation (RAG) repository-grounding setup for SLM-based traceability that reduces exhaustive pairwise comparisons by retrieving repository-local evidence, together with an analysis of how retrieval depth K governs the recall versus verification-effort trade-off.
- Practical, evidence-based implications for applying automated traceability methods in real GitHub repositories, highlighting how noise, scale, and model constraints (e.g., truncation and class imbalance) affect reliability and configuration choices.

1.5 Thesis Structure

The remainder of this thesis is organized as follows:

- Chapter 2 reviews existing literature on requirements traceability and traceability link recovery.
- Chapter 3 describes the study design, datasets, preprocessing and experimental setups used in this thesis.
- Chapter 4 presents and analyzes the experimental results.
- Chapter 5 discusses the implications, limitations, and practical relevance of the findings.
- Chapter 6 concludes the thesis and outlines future research directions.

2 Related Work

This chapter reviews research relevant to automated requirements traceability and Traceability Link Recovery (TLR), with a focus on linking issue reports to commits in GitHub repositories. The chapter begins with the foundations of software traceability and why link recovery is needed in practice. It then summarizes the main families of approaches that have been used for TLR, including Information Retrieval (IR), machine learning, and transformer-based models such as T-BERT. Because this thesis extends prior T-BERT-based work and evaluates a generative Small Language Model (SLM) with and without Retrieval-Augmented Generation (RAG), the chapter also introduces these concepts in an accessible manner and explains how they relate to traceability tasks.

2.1 Requirements Traceability and Traceability Link Recovery

Requirements traceability is the ability to establish and maintain explicit relationships between software development artifacts across the lifecycle, such as requirements, design decisions, source code, tests, and issue reports [15]. Traceability supports important software engineering activities, including impact analysis, maintenance, change management, and compliance verification [6].

In practice, however, trace links are often missing or incomplete. Maintaining traceability requires continuous effort from developers, and it is frequently treated as secondary to feature development and bug fixing [34]. Empirical studies report that traceability links in real projects are often inconsistent, outdated, or absent [24, 3]. This is particularly visible in GitHub projects, where issue-to-commit links depend heavily on whether developers consistently reference issue identifiers in commit messages.

To address missing links, research has focused on *Traceability Link Recovery (TLR)*, which aims to reconstruct trace links automatically (or semi-automatically) after development has occurred. In the context of this thesis, TLR refers to recovering links between GitHub issues and commits, enabling developers to better understand why changes were made and which issue context they relate to.

2.2 Information Retrieval Approaches to Traceability

Early automated traceability approaches relied primarily on Information Retrieval (IR). These methods treat artifacts as text documents and rank candidates based on lexical similarity [2]. The Vector Space Model (VSM) represents documents as

term vectors and typically uses cosine similarity to measure closeness [40]. TF-IDF weighting further improves retrieval by emphasizing terms that are specific to a document collection [25].

Latent Semantic Indexing (LSI) extends VSM by reducing dimensionality via Singular Value Decomposition, enabling similarity matching beyond exact token overlap [8]. LSI has been applied successfully in software traceability, such as linking requirements to code [26].

While IR-based approaches demonstrate that link recovery is feasible, they struggle in software engineering settings where vocabulary mismatch is common. Issue reports often describe behavior in natural language, while commits and code use technical identifiers, abbreviations, and implicit context. This mismatch can severely reduce lexical similarity even when artifacts are truly related [1, 11]. These limitations motivated the shift toward learning-based approaches.

2.3 Learning-Based Approaches

Machine learning approaches treat traceability recovery as a prediction or ranking problem: given an issue and a set of candidate commits, the model learns to assign higher scores to the commits that are truly linked [35]. Early work relied on *hand-crafted features*, that is, manually designed properties extracted from the candidate artifacts and from their relationship. In issue-commit traceability, such features may include lexical overlap between issue text and commit messages, similarity between identifiers appearing in the issue and in the changed code, temporal proximity, file-level metadata, and repository signals such as explicit issue references. These features are then used as input to classical classifiers or ranking models. Although such methods improved over pure IR baselines, their performance depended heavily on feature engineering and did not generalize well across projects and domains [35].

A key step beyond handcrafted features is *representation learning*. Rather than requiring researchers to manually specify which artifact properties may be useful, representation learning methods learn dense numerical vectors (*representations* or *embeddings*) directly from data, such that artifacts with similar meaning or function are placed closer together in the learned space. In traceability, this is attractive because issue reports, commit messages, and code changes may be related even when they do not share the same vocabulary.

Word embeddings such as Word2Vec and GloVe capture semantic similarity beyond exact token overlap [28, 33]. In software engineering, embedding-based representations have been used to support tasks such as bug localization and artifact linking [45]. However, earlier embedding approaches often represented words or artifacts in relatively shallow ways and struggled to fully capture the context of longer documents and complex code changes.

2.3.1 Deep Learning Models

Deep learning models (e.g., RNNs, LSTMs, GRUs) were introduced to better model sequence structure and context [18, 5]. They improved semantic modeling compared to shallow classifiers and IR techniques. For example, Guo et al. showed that deep neural representations can outperform IR baselines in traceability settings by learning richer semantic features [16].

A persistent challenge, however, is that supervised deep learning generally requires substantial labeled training data. In traceability, labeled links are scarce because ground truth often depends on developer discipline and manual validation effort [29]. This limitation contributed to the adoption of pre-trained language models, which can be adapted to new tasks with less labeled data.

2.4 Transformer-Based Traceability Models

Pre-trained transformer-based language models enabled major advances in traceability link recovery by providing stronger text and code representations that can be fine-tuned for specific tasks. A *transformer* is a neural network architecture introduced by Vaswani et al. [42] that models relationships between tokens using a self-attention mechanism. Unlike earlier sequential models such as RNNs or LSTMs, transformers process all tokens in parallel and learn which parts of the input are most relevant to each other. This makes them effective for capturing long-range dependencies in both natural language and source code.

Transformer models are commonly grouped into *encoder-only*, *decoder-only*, and *encoder-decoder* architectures. Encoder-only models, such as BERT, are designed to build contextual representations of the input. They read the entire input sequence and produce embeddings that are useful for tasks such as classification, ranking, and similarity estimation [10]. In traceability, this is a natural fit when the goal is to compare two artifacts and predict how strongly they are related.

Decoder-only models, in contrast, are generative models. They predict the next token given the preceding context, which allows them to generate text autoregressively. This family includes instruction-following language models that can answer questions, produce explanations, or make prompted decisions. In traceability, a decoder-only model can be asked directly whether an issue and a commit are linked, turning the task from similarity scoring into an explicit decision-making problem.

A third category, encoder-decoder models, combines both components and is often used for sequence-to-sequence tasks such as translation or summarization. Although such models are important in NLP more broadly, they are not the main focus of this thesis.

BERT is a widely used encoder-only example that introduced a powerful pre-training and fine-tuning paradigm for downstream applications [10]. Because source code differs from natural language, code-aware pre-trained models were introduced. CodeBERT is trained on paired natural language and programming language data from CodeSearchNet and is designed to bridge code and text representations [12, 20]. These models provide a natural foundation for issue–commit traceability.

2.4.1 T-BERT and Its Practical Input Limitations

Lin et al. proposed T-BERT for recovering issue–commit links [23]. T-BERT formulates traceability recovery as a ranking task: for a given issue, it ranks candidate commits so that true links appear near the top of the list. This formulation aligns with practical workflows, where developers prefer a short ranked list rather than a binary decision.

T-BERT Training Strategy

A defining characteristic of T-BERT is its multi-phase training strategy, which is designed to progressively align natural language artifacts and source code before performing traceability-specific learning [23]. Rather than training a model from scratch, T-BERT builds upon existing pre-trained models and introduces intermediate alignment stages to bridge the semantic gap between issues and commits. The training process consists of three main phases as shown in Figure 2.1:

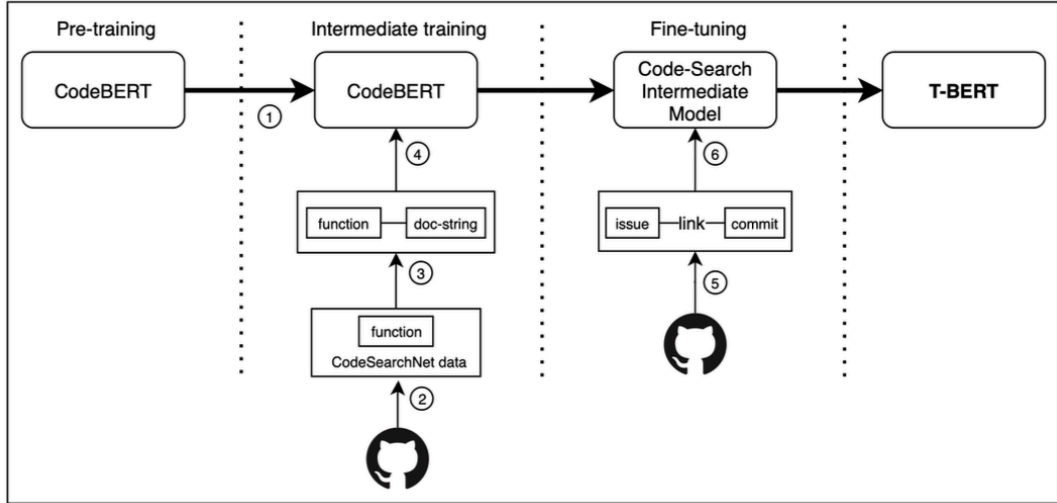


Figure 2.1 Multi-phase training strategy of T-BERT, consisting of code-aware initialization, intermediate alignment training, and traceability-specific fine-tuning

Phase 1: Pre-training T-BERT is initialized from CodeBERT, a pre-trained model trained on paired natural language and programming language data [12]. This

initialization provides a shared representation space for code and natural language, which is essential for linking issue descriptions to code changes.

Phase 2: Intermediate Training In the second phase, the model is trained on aligned function–documentation pairs which is extracted from CodeSearchNet dataset [20]. This step explicitly encourages the model to associate natural language descriptions with their corresponding code implementations, reducing the semantic gap between issues and commits.

Phase 3: Fine-Tuning In the final phase, the model is fine-tuned on issue–commit pairs for the traceability link recovery task. During this phase, the model learns to rank candidate commits for a given issue such that relevant commits appear higher in the ranked list. To improve discrimination between linked and non-linked artifact pairs, T-BERT uses *online negative sampling* (ONS), in which negative examples are generated dynamically during training. This strategy enables the model to encounter a broader range of challenging non-matching pairs and supports more effective traceability-specific learning.

This multi-phase training design allows T-BERT to leverage large-scale pre-training while adapting effectively to the traceability domain, and it forms the basis for many subsequent transformer-based traceability studies.

Although T-BERT demonstrates strong performance in controlled settings, subsequent real-world evaluations [30] highlighted limitations that matter in practice. A key limitation relevant to this thesis is the *loss of ordering in the input representation of code changes*. In the T-BERT pipeline, commit diffs are converted into plain text without preserving the original sequence of modified lines. As a result, code changes may be processed as an unordered collection rather than as a structured, sequential modification.

This loss of ordering removes important contextual information about how a change develops within the code, which is especially problematic for commits containing multiple related edits or refactorings. Although BERT-based models are also subject to additional constraints, such as the size of the model’s *context window*, which limits the amount of input that can be processed simultaneously, such limitations are not central to this thesis. Instead, RQ1 is concerned specifically with preserving and restoring the sequential structure of code changes.

By making this input limitation explicit, this section motivates the input restructuring strategies evaluated later in the thesis for the T-BERT baseline.

2.4.2 Other Transformer-Based Models

Following T-BERT, other transformer-based models introduced refinements in how issue and commit information is encoded and, importantly, how the scarcity of labeled traceability data is addressed. In real-world software projects, manually

curated issue–commit links are often limited, incomplete, or noisy, which poses a significant challenge for supervised learning approaches.

BTLink uses a BERT-based setup for issue–commit linking [21], primarily focusing on improving representation learning, but still relying on labeled pairs for effective training. To mitigate the dependency on labeled data, DSSLink introduces a semi-supervised learning framework that leverages large amounts of unlabeled artifact pairs [47]. By incorporating pseudo-labeling and consistency-based training objectives, DSSLink is able to extract useful training signals from data that would otherwise remain unused.

Similarly, MTLLink explores a multi-task learning strategy to improve robustness under limited supervision [9]. By jointly learning related tasks, such as auxiliary classification or similarity estimation objectives, the model can share representations across tasks and generalize better even when labeled traceability links are scarce.

While these approaches differ in their strategies for coping with limited labeled data ranging from semi-supervised learning to multi-task optimization, they typically retain an encoder-based formulation in which the primary objective is to compute relevance or ranking scores between artifact pairs.

2.5 Small Language Models for Traceability

In addition to encoder-based models, an alternative direction is to use decoder-based models to perform traceability decisions through prompting. In this thesis, a *Small Language Model (SLM)* refers to a generative transformer-based language model that is substantially smaller than frontier-scale proprietary models, yet still capable of producing coherent natural language outputs and handling code-related inputs, and suitable for deployment under tighter computational and latency constraints [44].

A practical advantage of this approach is that traceability can be posed directly as a natural-language task: given an issue and a commit, the model is asked to judge whether they are related and to produce a label or explanation. This can be used in different supervision settings:

- **Zero-shot:** the model is given only task instructions and input examples at inference time, without any task-specific training [4, 38].
- **Few-shot:** the model is additionally provided with a small number of labeled examples inside the prompt to illustrate the desired behavior [4, 38].

While generative models are widely used in software engineering tasks such as code explanation and summarization [46], their behavior in traceability link recovery is less established than encoder-based models. In this thesis, the SLM is evaluated

exclusively in a zero-shot setting. The decision to focus on zero-shot prompting is deliberate: it isolates the model’s inherent capability to reason about issue–commit relationships without task-specific training examples or prompt engineering heuristics. This allows a clearer comparison with supervised encoder-based approaches such as T-BERT, while also reflecting realistic deployment scenarios where labeled examples may be scarce or unavailable. More advanced prompting strategies, such as few-shot learning, Chain-Of-Thought (CoT) reasoning, or self-consistency are not explored, as the goal of RQ2 is to assess the baseline traceability performance of a generative SLM under minimal task adaptation.

2.5.1 Rationale for Selecting Code Llama 7B Instruct

This thesis uses `Code Llama 7B Instruct` as the generative SLM [39]. The choice is motivated by several practical factors that are relevant for traceability experiments:

Code-oriented training. Code Llama is trained on code-focused corpora, which makes it more suitable for reasoning over commit messages and code changes than general-purpose text-only models [39].

Instruction-following behavior. The Instruct variant is tuned to follow natural language instructions, which is important when traceability is framed as a prompted decision task (e.g., “predict whether this issue and commit are related”) [39].

Longer input handling than BERT-based models. BERT-based models such as T-BERT are typically limited to 512 tokens, whereas Code Llama supports much longer contexts (up to 16,000 tokens [39]). This is relevant for traceability link recovery, as commit diffs can be large and contain many changed lines [37], which may exceed the 512-token limit and require truncation in BERT-based models.

Practical feasibility. Among Code Llama variants, the 7B model provides a balance between capability and computational cost. Larger variants (e.g., 13B, 34B) increase resource requirements substantially, making large-scale issue–commit evaluation more expensive. Selecting 7B supports experimentation at a realistic scale without relying on restricted proprietary models [39].

2.6 Retrieval-Augmented Generation and Repository Context

A recurring challenge in traceability link recovery is that artifacts are often incomplete when considered in isolation. In issue–commit traceability, commit messages may omit implementation details, and issue descriptions may lack precise references to the modified code. Relevant contextual information may instead reside elsewhere in the repository, such as related issues, prior commits, or file history.

Retrieval-Augmented Generation (RAG) addresses this limitation by combining information retrieval with generative language models [22]. In a RAG pipeline, relevant documents are first retrieved from an external corpus and then provided as additional context to the language model during inference. This allows the model to ground its predictions in repository-level information beyond the immediate artifact pair.

Recent work by Hey et al. [17] investigated RAG specifically for inter-requirements traceability link recovery. Their approach retrieves semantically similar requirement artifacts and uses large language models to classify trace links between requirement pairs. They demonstrate that a RAG-based approach can outperform several state-of-the-art baselines in requirement-to-requirement traceability.

While their work focuses on inter-requirements traceability and evaluates a dedicated RAG-based architecture on standardized datasets with known traceability links, this thesis investigates RAG in a different context. Specifically, this study applies RAG to issue–commit traceability in real GitHub repositories, where artifacts combine natural language and source code diffs. Rather than proposing a new retrieval framework, RQ3 evaluates how RAG-based repository grounding affects the behavior of a zero-shot Small Language Model. As a sub-analysis, the effect of retrieval depth (K) is examined to understand how retrieval design shapes the precision–recall and verification–burden trade-off. By contrasting RAG-augmented generation with both a zero-shot baseline and the encoder-based T-BERT model, this thesis provides empirical insights into the role of contextual grounding for issue–commit traceability under realistic repository conditions.

2.7 Summary and Research Gaps

Prior research demonstrates that automated traceability has progressed from lexical IR methods to learning-based and transformer-based approaches. Nevertheless, several gaps remain relevant to practical traceability in real repositories. First, although T-BERT is widely used as a strong baseline, the impact of its input construction especially loss of code-change ordering and structure is not well understood, motivating the input restructuring study in RQ1. Second, most traceability studies emphasize encoder-based models, while the traceability behavior of Small Language Models is underexplored, motivating RQ2. Third, repository-level grounding remains a key practical challenge. RAG offers a mechanism for incorporating repository-level context into traceability link recovery. However, its effect on traceability performance when used in combination with Small Language Models has not been systematically evaluated, motivating RQ3. Together, these gaps motivate the empirical investigations presented in the remainder of this thesis.

3 Research Methodology

This chapter describes the methodology used to investigate Traceability Link Recovery (TLR) in real-world GitHub repositories, building on the practical limitations observed in [30]. The goal throughout the work was to improve *precision* for automated traceability link recovery.

The experimental direction reported in this chapter emerged iteratively. We implemented an approach, analyzed its behavior on real repositories, and then decided on the next step based on what we observed. Thus, the resulting process is reported as three experiment blocks that share the same input artifacts.

We started from the T-BERT baseline and tested the input restructuring direction suggested as future work in [30]. Specifically, we restructured commit diffs to preserve sequential order and change direction while keeping the T-BERT training and evaluation protocol unchanged (see Section 3.2). After observing that restructuring did not improve performance in the expected way, we attributed much of the limitation to architectural constraints (see Section 4.1).

Given the representational and contextual limitations of the T-BERT architecture, it motivated us to transition to a Small Language Model (SLM), which would enable a more direct comparison of an issue and a commit to determine whether they are linked or not. We used the issue text, commit summary, and commit diff as input (see Section 3.3). However, the model often linked too many pairs, which increased false positives (Section 4.2).

Finally, we introduced Retrieval-Augmented Generation (RAG) to ground SLM decisions using repository local context and to avoid exhaustive pairwise prompting by retrieving a smaller set of relevant commit fragments per issue (see Section 3.4). Within RQ3, retrieval depth K is treated as the main experimental factor (RQ3.1).

Across all experiments, predictions are evaluated against expert-labeled ground truth from Parveen et al. [31, 32], using the metrics defined in Section 3.5. Results are presented in Chapter 4 and discussed in Chapter 5.

3.1 Input Artifacts Definition

The input artifacts in all experiments are **Issue–Commit** pairs. Each pair is converted into a textual representation that can be consumed by the evaluated model: (i) as a transformer input for T-BERT, or (ii) as prompt content for the SLM/RAG experiments. Using the same input artifact across experiments supports consistent comparison across RQ1–RQ3.

3.1.1 Issue Artifact

An **Issue Artifact** is constructed from:

- **Title**
- **Description** (cleaned)

Cleaning removes formatting that can dominate the token budget without adding useful signal for traceability (e.g., markdown fences, HTML comments, repeated templates, and very long stack traces). The same issue construction is used across all the experiments.

3.1.2 Commit Artifact

A **Commit Artifact** consists of:

- **Commit summary message**
- **Commit diff** (changed lines extracted from `git diff`)

For RQ1, the commit diff representation is the manipulated component. For RQ2–RQ3, the commit artifact is included in the prompt in “commit summary + commit diff” form. Figure 3.1 illustrates how raw GitHub artifacts are normalized into the text representations used by the evaluated models.

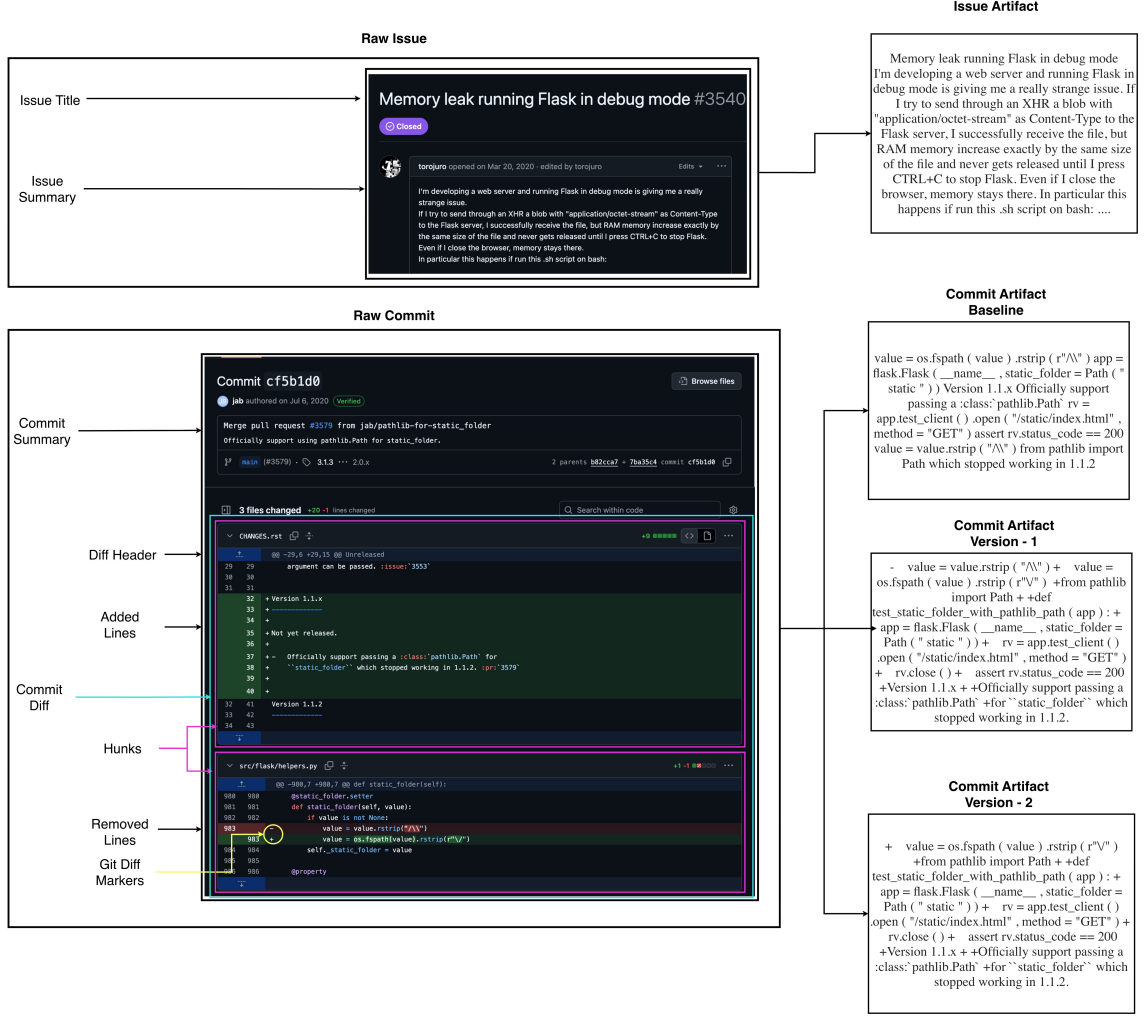


Figure 3.1 Example of artifact construction used to build Issue and Commit artifacts

3.1.3 Commit Diff Normalization Across Model Families

Although all experiments operate on Issue–Commit pairs, commit diffs are represented differently across model families due to architectural constraints.

T-BERT (RQ1). Following the T-BERT preprocessing approach in [23], commit diffs are transformed into a compact plain-text format by removing diff headers and other non-essential metadata. In RQ1, the restructuring of this normalized input is designed to preserve the order of changes and their direction (for example, whether content was added or removed), rather than to keep the complete commit diff.

SLM and RAG (RQ2–RQ3). For the other two experiments, the git diff markers and hunk structure (i.e., contiguous blocks of changes in a diff) are preserved in the commit artifact. In practice, symbols such as addition (+) and deletion (–) provide explicit cues about what changed, which may be useful when the task is posed

as a prompted decision. For these experiments, only the *issue artifact* is cleaned to reduce noise, while commit diffs are kept closer to their `commit diff` form.

These preprocessing differences are intentional and reflect how each model family is typically used. T-BERT is evaluated under its established preprocessing pipeline to remain comparable with prior work, and RQ1 isolates the effect of *restructuring* within that pipeline. In contrast, the SLM is evaluated as a prompted decision maker, where preserving the entire commit diff structure (including +/- and hunk boundaries) is a reasonable representation of how these models view code changes [19].

3.2 RQ1: T-BERT Commit Diff Restructuring Experiment

This section describes the experimental setup used to answer RQ1. We keep the original T-BERT training and evaluation protocol fixed and vary only the commit diff representation, to isolate the effect of diff ordering and change-direction cues under the standard preprocessing and fixed input-length constraints.

3.2.1 Effect of Commit Diff Ordering and Input Construction

To illustrate the practical implications of commit diff flattening mentioned in [30], we present a realistic example from the Flask repository. Figure 3.2 shows the original GitHub diff view for the example commit. From this raw representation, T-BERT preprocessing removes file headers, hunk metadata, and most diff-specific formatting, producing a flattened textual artifact. The commit restores support for passing a `pathlib.Path` object to the `static_folder` parameter. The change modifies helper logic, adds a regression test, and updates the changelog.

```

CHANGES.rst
@@ -29,6 +29,15 @@ Unreleased
29 29     argument can be passed. :issue:`3553`
30 30
31 31
32 + Version 1.1.x
33 + -----
34 +
35 + Not yet released.
36 +
37 + - Officially support passing a :class:`pathlib.Path` for
38 +   ``static_folder`` which stopped working in 1.1.2. :pr:`3579`
39 +
40 +
32 41  Version 1.1.2
33 42  -----
34 43

src/flask/helpers.py
@@ -980,7 +980,7 @@ def static_folder(self):
980 980     @static_folder.setter
981 981     def static_folder(self, value):
982 982         if value is not None:
983 -             value = value.rstrip("/\\")
983 +             value = os.fspath(value).rstrip(r"\\")
984 984         self._static_folder = value
985 985
986 986     @property

tests/test_basic.py
@@ -1406,6 +1406,16 @@ def test_static_url_empty_path_default(app):
1406 1406     rv.close()
1407 1407
1408 1408
1409 + @pytest.mark.skipif(sys.version_info < (3, 6), reason="requires Python >= 3.6")
1410 + def test_static_folder_with_pathlib_path(app):
1411 +     from pathlib import Path
1412 +
1413 +     app = flask.Flask(__name__, static_folder=Path("static"))
1414 +     rv = app.test_client().open("/static/index.html", method="GET")
1415 +     assert rv.status_code == 200
1416 +     rv.close()
1417 +
1418 +
1409 1419     def test_static_folder_with_ending_slash():
1410 1420         app = flask.Flask(__name__, static_folder="static/")
1411 1421

```

Figure 3.2 Example of a commit diff from flask repository

Unordered diff representation In the baseline preprocessing pipeline used by T-BERT, hunk boundaries and file separation are removed, and change markers (+/-) are discarded. Lines originating from different files and logical edits may therefore become interleaved:

```

value = os.fspath ( value ) .rstrip ( r"\\/\" )
app = flask.Flask ( __name__ , static_folder = Path ( " static " ) )
Version 1.1.x
Officially support passing a :class:`pathlib.Path`
rv = app.test_client ( ) .open ( "/static/index.html" , method = "GET" )
assert rv.status_code == 200
value = value.rstrip ( "/\\" )
from pathlib import Path
which stopped working in 1.1.2

```

In the above representation, the removal of `value.rstrip("/\\")` and its replacement with `os.fspath(value)` are no longer explicitly connected. Additions to tests and documentation are mixed with helper logic modifications, and the direction of change is lost. As a result, it becomes difficult for the model to distinguish which lines were removed and which were added, and to determine how the edits are related across files.

Ordered diff representation In the proposed restructuring strategy, the original sequential ordering of the diff is preserved, and the direction markers (+/-) are retained. However, file headers and hunk boundaries are not included. The resulting representation appears as follows:

```
-    value = value.rstrip ( "/"\\ " )
+    value = os.fspath ( value ) .rstrip ( r"\/" )

+from pathlib import Path
+
+def test_static_folder_with_pathlib_path ( app ) :
+    app = flask.Flask ( __name__ , static_folder = Path ( " static " ) )
+    rv = app.test_client ( ) .open ( "/static/index.html" , method = "GET" )
+    rv.close ( )
+    assert rv.status_code == 200

+Version 1.1.x
+
+Officially support passing a :class:`pathlib.Path`
+for ``static_folder`` which stopped working in 1.1.2.
```

In this representation, the removed line and its corresponding replacement remain adjacent, preserving the local edit relationship. Subsequent additions, including the regression test and changelog update, appear in the same relative order as in the original commit. Although explicit file boundaries are not retained, the sequential structure and direction of edits are preserved.

Input Construction Formally, the ordered diff representation is constructed as follows:

1. **Parse diff into hunks:** split the `git diff` into file-level change regions and remove diff metadata (like `diff --git`, file path headers, and `@@` line markers, since they don't provide any semantic meaning) so that only the added and removed lines remain.

2. **Preserve sequential order:** process lines in the order emitted by `git diff`.
3. **Assemble commit input:** concatenate commit summary + ordered diff.

Preserving sequential order and edit direction, therefore, retains structural signals that may assist the model in aligning issue descriptions with the corresponding code modifications during fine-tuning.

3.2.2 Input Configurations for T-BERT

Two input configurations are evaluated to investigate how different representations of code changes affect traceability learning. Version 1 preserves both additions and deletions to maintain the full change context, whereas Version 2 retains only added lines to assess whether focusing on newly introduced code can reduce noise and emphasize the most relevant parts in a commit.

- **Version - 1:** commit summary + ordered diff (additions and deletions retained)
- **Version - 2:** commit summary + ordered diff (only added lines retained)

Figure 3.3 illustrates the Phase 3 fine-tuning pipeline for RQ1. Starting from the Flask training dataset, two commit artifact representations (V1 and V2) are constructed, and the same Phase 3 fine-tuning procedure is applied to obtain two T-BERT models that differ only in the commit diff input format.

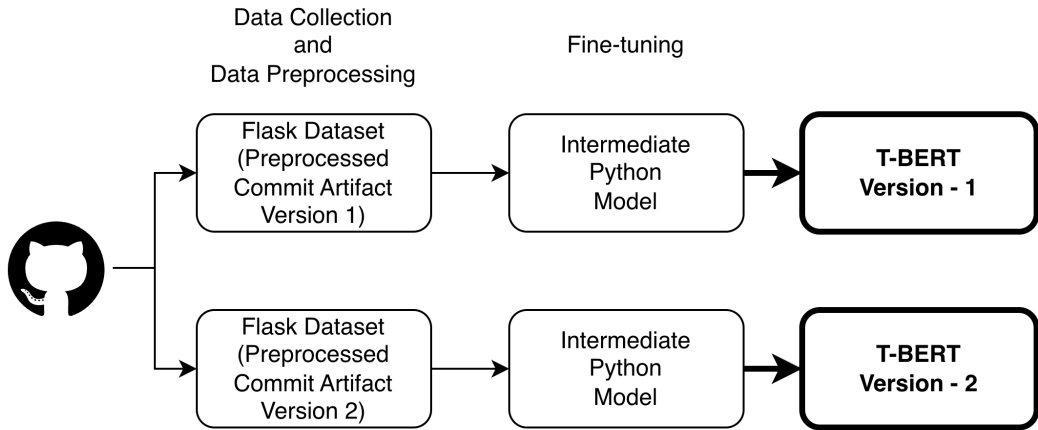


Figure 3.3 Fine-tuning setup for T-BERT using two commit artifact variants (V1 and V2)

As T-BERT is based on a BERT-style encoder with a fixed maximum sequence length (512 tokens), commit representations that exceed this limit are automatically truncated by the tokenizer during preprocessing. Truncation follows the default

behavior of the T-BERT implementation, where excess tokens beyond the maximum sequence length are removed. This behavior applies consistently across the baseline and both ordered-diff variants (V1 and V2).

3.2.3 Training and Cross-Project Evaluation Datasets

Fine-tuning follows the T-BERT protocol, including online negative sampling [23] (see Section 2.4.1). Both variants (V1 and V2) are fine-tuned on the **Flask** dataset and evaluated in a cross-project setting on two unseen repositories (**Keras** and **Pgcli**). The repositories used for Phase 3 fine-tuning and cross-project testing are presented in Table 3.1.

Table 3.1 Repositories used for T-BERT fine-tuning and cross-project evaluation

Dataset	GitHub Repository	Language	Issues	PRs	Commits
Flask	pallets/flask ¹	Python	2674	2635	5411
Keras	keras-team/keras ²	Python	12425	7956	11449
Pgcli	dbcli/pgcli ³	Python	713	771	2383

3.3 RQ2: Zero-Shot Traceability With a SLM

RQ1 examined whether changing the structure of commit diffs could improve T-BERT. However, even with better diff formatting, T-BERT still remains limited by its architecture and maximum input length, especially when commits are large (see Section 4.1 and Section 5.1). For this reason, RQ2 moves to a Small Language Model (SLM), which takes a prompt containing the issue text, commit summary, and commit diff, and then makes a direct decision about whether the issue and commit are linked. This allows the traceability task to be studied as an explicit decision problem rather than only as similarity scoring. The results of this approach, including precision, recall, and false-positive behavior compared to T-BERT, are presented in Chapter 4 and discussed in Chapter 5.

3.3.1 Model Selection and Zero-Shot Setting

We evaluate **CodeLlama-7B-Instruct** [39] as a SLM. In this thesis, **zero-shot** means: (i) no traceability specific fine-tuning, and (ii) no labeled examples included

¹<https://github.com/pallets/flask>

²<https://github.com/keras-team/keras>

³<https://github.com/dbcli/pgcli>

in the prompt. The model receives task instructions, an Issue Artifact, and a Commit Artifact, and outputs a binary decision (**True/False**). A second prompt variant additionally requests a confidence score (0–100). The prompt templates used in these experiments are provided in Appendix A. It is important to note that this thesis does not explore prompt engineering or prompt optimization. The prompts are intentionally kept simple and fixed across experiments to illustrate a plausible zero-shot application scenario.

3.3.2 Model Configuration

To make SLM outputs reproducible and comparable across datasets and experimental settings, inference is configured to be deterministic. The decoding configuration disables sampling and fixes generation randomness so that identical Issue–Commit inputs yield the same outputs across runs. Table 3.2 summarizes the parameters used for the chosen SLM.

Table 3.2 *Generation parameters used for deterministic zero-shot inference with the SLM*

Parameter	Value	Description
padding	True	Pads sequences to the maximum length supported by the tokenizer.
truncation	True	Truncates sequences exceeding the context window limit.
do_sample	False	Disables sampling.
temperature	0.0	Ensures deterministic, non-random output.

3.3.3 Data Acquisition and Preprocessing

Issues, commits, and diffs are collected using the GitHub REST API and GitPython [13, 14]. Artifacts are constructed as defined in Section 3.1.1 and Section 3.1.2. The git diff markers and hunk structure are preserved in the Commit Artifact for the SLM experiments (Section 3.1.3). Data is stored in Parquet for reproducibility and efficient processing. The evaluation repositories for RQ2 and RQ3 are presented in Table 3.3.

3.3.4 Execution Protocol

For each Issue–Commit pair:

1. Insert the Issue Artifact and Commit Artifact into the fixed prompt template.

2. Run inference with deterministic decoding.
3. Parse the output into a binary label (and confidence score for prompt template 2).
4. Store predictions alongside the expert ground truth label.

This process is illustrated in Figure 3.4.

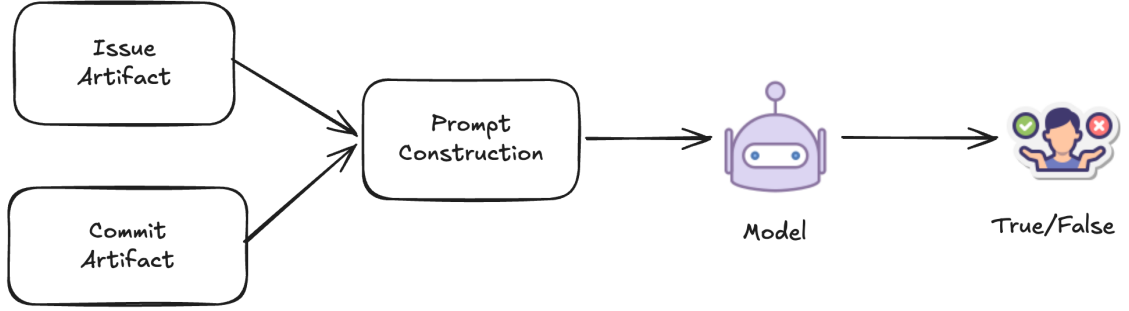


Figure 3.4 Zero-shot prompting workflow for the generative SLM

The zero-shot SLM acts as a binary classifier over Issue–Commit pairs. For each dataset, we run inference once per available pair and store the predicted label (and confidence, when requested) alongside the expert reference label. This protocol ensures that all RQ2 results reflect the same unit of prediction and the same prompt template, enabling consistent comparisons across repositories. The output behavior is analyzed in Chapter 4 and interpreted in Chapter 5.

3.4 RQ3: Repository-Grounded Traceability via RAG

Retrieval-Augmented Generation (RAG) combines information retrieval with a small language model by retrieving external context at inference time and inserting it into the model prompt [22]. In this study, RAG is used to ground traceability decisions using repository-specific artifacts by allowing the SLM to access semantically related commit fragments from the same repository when evaluating a given Issue artifact. The RAG pipeline in this thesis consists of (i) repository context preparation and indexing, (ii) retrieval for a given issue query, and (iii) prompt construction with retrieved context.

While RQ2 evaluates the SLM as a decision maker over Issue–Commit pairs, two practical limitations motivate RQ3. First, evaluating all $|Issue| \times |Commit|$ pairs can be computationally expensive in larger repositories. Second, prompting the model with a single commit at a time provides limited repository-wide grounding, which can make it difficult to judge whether a commit is the *best* match for an issue

when many commits are semantically similar. RQ3 therefore, introduces Retrieval-Augmented Generation (RAG) to retrieve a small set of semantically related commit fragments per issue and inject them as evidence into the prompt. This design both grounds decisions and reduces exhaustive pairwise comparisons. The resulting trade-offs are examined in Chapter 4 and Chapter 5.

3.4.1 Repository Context Preparation and Indexing

In this study, repository context or repository grounding refers to historical development artifacts extracted from the same github repository that may provide evidence for linking an issue to a commit.

Chunking Commit diffs can exceed practical prompt limits, and indexing full commits reduces retrieval granularity. We therefore split each commit artifact (summary + diff) into smaller overlapping **chunks**, so that retrieved context can be more precise and still fit within the SLM prompt.

Chunking parameters Each commit document is split into chunks of up to 3000 tokens with a 200-token overlap. Each chunk is stored with metadata (commit SHA, timestamp, author name, and repository). These values are chosen to ensure that each chunk contains sufficient code in context window while keeping retrieved context small enough to fit within the SLM prompt alongside the issue and task instructions.

Embedding and vector index Each chunk is converted into a dense vector using the **Embedding Gemma** model [43] and stored in a **ChromaDB** vector index, which supports efficient nearest-neighbor search over embedded repository chunks.

The same embedding model is used to represent both issue text and commit chunks in a shared vector space, enabling semantic retrieval even when issues and commits use different terminology. ChromaDB performs nearest-neighbor search over these vectors, and cosine similarity is used to rank retrieved chunks by similarity [28, 25].

3.4.2 Evaluation Repositories

Unlike RQ1, which follows the original T-BERT experimental setup for comparability with prior work, RQ2 and RQ3 are evaluated on a different set of repositories. This choice reflects the different purposes of the experiments. For RQ1, we retained the Flask-based training and cross-project evaluation protocol established in earlier work so that any observed effect could be attributed specifically to the proposed

commit-diff restructuring rather than to changes in datasets or evaluation conditions.

For RQ2 and RQ3, the objective shifts from reproducing the T-BERT setting to studying zero-shot and repository-grounded traceability in a more realistic repository context. For these experiments, we selected repositories for which expert-validated traceability links were already available from prior work [30]. This provides a reliable reference set for evaluating prompted SLM decisions and RAG-based retrieval behavior on real repository artifacts. In addition, because these validated links were established in earlier T-BERT-based studies [31, 30], their use also facilitates comparison with prior validation results. Table 3.3 summarizes the repositories used to evaluate the SLM and the RAG pipeline, including the resulting number of indexed chunks.

Table 3.3 *Summary of evaluation repositories used for SLM and RAG experiments (RQ2 and RQ3)*

Dataset	GitHub Repository	Lang.	Issues	PRs	Commits	Chunks	Total Pairs
Test Project	test-project ¹	Python	5	2	9	9	45
EV-Components	GUI ²	JS	7	39	63	164	441
Liquid AI-Supervisor	wasmiot-supervisor ³	Python	19	21	253	670	4807

3.4.3 Retrieval and Prompt Construction

Given an issue, the preprocessed Issue Artifact is used as the retrieval query:

1. The issue artifact is given as input to the system which is embedded into vector by the embedding model.
2. The system retrieves the top- K most similar commit chunks with respect to the embedded issue vector using cosine similarity.
3. Constructs a RAG prompt that includes: (i) task instructions, (ii) the issue text, and (iii) the retrieved chunks as repository context. The prompt template used in this approach is given in Appendix A.
4. Run `CodeLlama-7B-Instruct` and record the output.

As shown in Figure 3.5, retrieval operates over embedded chunks rather than full commits, enabling scalable context injection under prompt-length constraints.

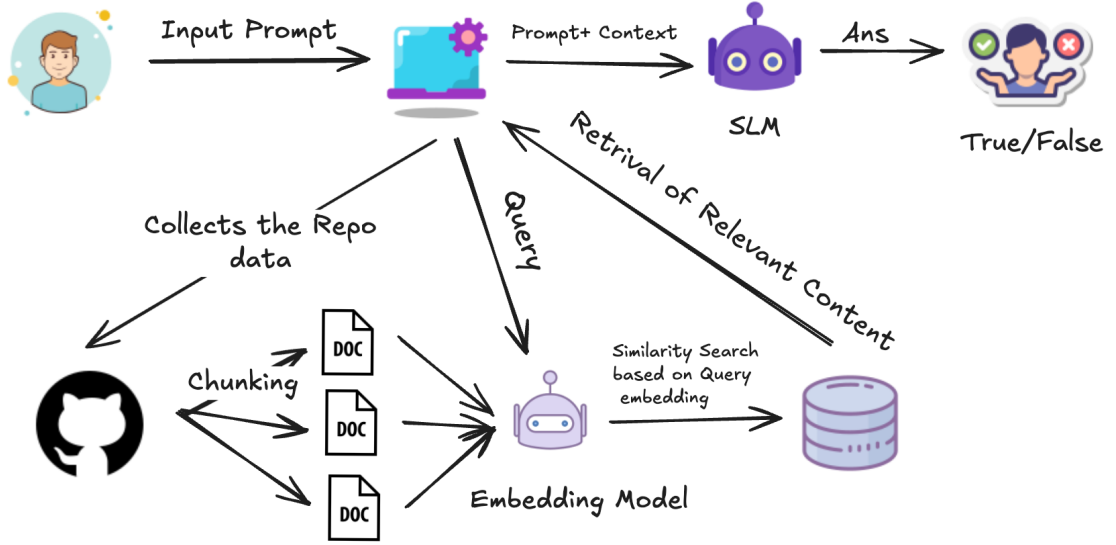


Figure 3.5 RAG pipeline for repository-grounded traceability decisions

3.4.4 Retrieval Depth as an Experimental Factor

To study how retrieved context affects traceability decisions (RQ3.1), we vary retrieval depth $K \in \{1, 5, 10, 15, 20, 30\}$, where K controls how many commit chunks are inserted into the prompt. Intuitively, a larger K shows the model more potentially relevant evidence, but it also increases the chance of including unrelated fragments.

In this thesis, we vary *only* K to keep the comparison simple and attributable as changing chunk size, overlap, embedding model, or indexing settings would also change *what* gets retrieved and *how much* retrieved content fits into the prompt. If multiple retrieval parameters were changed at the same time, it would be unclear whether observed differences come from retrieval depth or from a different chunking/representation. Therefore, all other retrieval components are held constant and only K is treated as the experimental factor. Performance is compared across K values (RQ3.1) and contextualized against the baseline T-BERT in Chapter 4, then discussed in Chapter 5.

3.5 Evaluation Protocol and Metrics

3.5.1 Mapping Reference Labels

The reference dataset assigns four labels to Issue–Commit pairs: **True**, **Partially True**, **Unclear**, and **False**. Following prior work [30, 32], we map these into binary classes:

- **Positive:** True and Partially True

- **Negative: False and Unclear**

This mapping is required because the evaluated systems produce binary decisions. Pairs without expert labels are treated as negative for the purpose of metric computation. However, this assumption may introduce label noise, as some true links may be missing from the reference data; this limitation is discussed in Chapter 5.

3.5.2 Classification Metrics: Precision and Recall

For pairwise predictions (RQ2–RQ3) we report **precision** and **recall** computed from the confusion matrix. Each prediction belongs to one of four categories:

- **True Positive (TP):** The model predicts a link and the link is correct.
- **False Positive (FP):** The model predicts a link but the link is incorrect.
- **False Negative (FN):** The model failed to predict a true link.
- **True Negative (TN):** The model predicts a non link.

Precision measures the proportion of predicted links that are correct:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3.1)$$

Recall measures the proportion of true links that are successfully recovered:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3.2)$$

Precision is important because false positives increase manual verification effort, while recall is important because missing links reduce traceability completeness.

3.5.3 Ranking Metrics: Precision@K, MAP@3, and MRR

T-BERT produces, for each issue, a ranked list of candidate commits. Evaluation must therefore measure not only whether correct commits are retrieved, but also how highly they are ranked. Since developers typically inspect only the first few results, early ranking quality is critical. Formally, let Q denote the set of all issues in the evaluation dataset.

Precision@K It measures the proportion of relevant commits among the top K ranked results:

$$\text{P@K} = \frac{\text{Number of relevant commits within top } K}{K} \quad (3.3)$$

It reflects the quality of the first K recommendations.

Mean Reciprocal Rank (MRR) It measures how early the first relevant commit appears in the ranking. For each issue, we compute:

$$\text{ReciprocalRank}_q = \frac{1}{\text{rank}_q} \quad (3.4)$$

where rank_q is the position of the first relevant commit. MRR is the average of these values over all issues:

$$\text{MRR} = \frac{1}{|Q|} \sum_{q \in Q} \frac{1}{\text{rank}_q} \quad (3.5)$$

MRR emphasizes how quickly the first correct link appears.

Mean Average Precision (MAP@3) It evaluates how well relevant commits are ranked within the top three positions. For each issue, Average Precision at 3 is computed as:

$$\text{AP@3}(q) = \frac{1}{\min(R_q, 3)} \sum_{i=1}^3 \text{P@i}(q) \cdot \text{rel}_q(i) \quad (3.6)$$

where:

- R_q is the number of relevant commits for issue q ,
- $\text{P@i}(q)$ is precision at rank i ,
- $\text{rel}_q(i)$ is 1 if the commit at rank i is relevant, and 0 otherwise.

MAP@3 is the mean of $\text{AP@3}(q)$ over all issues:

$$\text{MAP@3} = \frac{1}{|Q|} \sum_{q \in Q} \text{AP@3}(q) \quad (3.7)$$

In summary:

- Precision@K measures top- K recommendation quality.
- MRR measures how early the first relevant result appears.
- MAP@3 measures whether relevant commits are consistently ranked within the top three.

3.5.4 Metric Usage Across RQ1–RQ3

We report metrics as follows:

- **RQ1:** MAP@3, MRR, precision, confusion matrix, and recall following the T-BERT evaluation protocol [23].
- **RQ2:** Precision, recall, and confusion matrix per dataset.
- **RQ3:** Precision and recall for the RAG-grounded setup, analyzed as a function of retrieval depth K (RQ3.1).

4 Results

This chapter reports the results for the three research questions introduced in Chapter 1. We first investigate how alternative commit input restructuring strategies affect traceability link recovery (TLR) performance in the T-BERT baseline (RQ1). We then compare the performance of Small Language Model (SLM) against the T-BERT baseline (RQ2). Finally, we evaluate the impact of Retrieval-Augmented Generation (RAG)-based repository grounding on SLM-based traceability and analyze how retrieval depth K shapes the resulting trade-offs (RQ3, RQ3.1).

4.1 RQ1: Impact of Commit Input Restructuring on T-BERT Performance

This section evaluates whether restructuring commit diffs to preserve sequential order and change direction improves the performance of T-BERT. We compare the original T-BERT input representation [23] (Baseline) against two ordered-diff variants introduced in Chapter 3: **Version 1** (ordered diff with additions and deletions) and **Version 2** (ordered diff with additions only). Following the standard ranking-based evaluation protocol for T-BERT mentioned in Section 3.5.3, we report MAP@3 and MRR.

4.1.1 Cross-Project Results on Benchmark Repositories

Table 4.1 summarizes performance when all models are fine-tuned on *Flask* and evaluated cross-project on *pgcli* and *keras*.

Table 4.1 *T-BERT Ranking Performance with Confusion Matrix Statistics*

Test Split	Model	MAP@3	MRR	Precision	Recall	TP	FP	TN	FN
pgcli	Baseline	0.847	0.858	0.854	0.672	41	7	3592	20
	Version 1	0.826	0.839	0.809	0.623	38	9	3590	23
	Version 2	0.833	0.845	0.830	0.639	39	8	3591	22
keras	Baseline	1.000	1.000	0.930	0.964	53	4	2966	2
	Version 1	0.965	0.987	0.895	0.927	51	6	2964	4
	Version 2	0.944	0.966	0.875	0.891	49	7	2963	6

Key observation Across both evaluation repositories, the Baseline outperforms both ordered-diff variants on MAP@3 and MRR. The decrease is modest on *pgcli*, but more visible on *keras*, where the baseline achieves perfect scores while ordered variants reduce MAP@3 and MRR.

Interpretation The goal of the restructuring was to strengthen the signal available to T-BERT by preserving the original diff order and explicitly retaining $+/-$ markers to indicate code additions and deletions. We assumed that these structural cues would help the model better connect implementation changes to issue descriptions and, consequently, improve the performance.

However, the results do not support this assumption. Across both repositories, MAP@3 and MRR decrease for the ordered-diff variants, indicating that the first correct link appears later in the ranking (MRR) and that relevant commits are less consistently placed within the top three positions (MAP@3). This decrease is caused by changes in the prediction scores assigned by the model to specific issue-commit pairs, which push the correct commits lower in the ranking. Under the same evaluation protocol, the confusion matrix statistics provide an additional view of what changed in the model behavior.

The precision decreases for both ordered variants in both datasets (from 0.854 to 0.809 on *pgcli*, and from 0.930 to 0.895 on *keras*). This corresponds to a small increase in false positives (FP), meaning the model is more likely to assign a high rank to commits that are not truly linked to the issue. From a practical perspective, higher FP implies more incorrect candidates among the top recommendations, increasing the manual verification burden for developers.

Second, recall also decreases alongside a rise in false negatives (FN) (FN increases from 20 to 23 on *pgcli*, and from 2 to 4 on *keras* for Version 1). This indicates that some true links are pushed lower in the ranked list or missed within the evaluation cutoff, which is consistent with the observed drop in MAP@3 and MRR. In other words, the restructuring not only fails to reduce the false positives (FP), but it also slightly reduces the model’s ability to surface true links early.

Overall, the ordered diff representations introduce additional structural cues (sequential ordering and change direction) compared to the baseline preprocessing. However, these cues do not appear to translate into additional semantic evidence that the T-BERT architecture can effectively leverage during fine-tuning. In practice, the restructuring modifies the input format but does not improve the model’s ability to align issue descriptions with the corresponding code changes. This aligns with prior observations by Parveen et al. [30] that T-BERT behavior is dominated by semantic cues present in the text, while structural or surface-level formatting changes alone are insufficient to yield measurable gains under fixed context-length constraints. In the evaluated setting, ordered diff restructuring does not improve T-BERT and yields performance comparable to, but below, the original baseline.

4.2 RQ2: Traceability Performance of a SLM Compared to T-BERT

This section evaluates `CodeLlama-7B-Instruct` under a zero-shot prompting setup and compares its binary traceability decisions to the T-BERT baseline [30].

4.2.1 Results Across Evaluation Projects

Table 4.2 report the outcomes for T-BERT and the SLM using two prompt templates. The SLM achieves perfect recall on all three projects, but at the cost of substantially increased false positives, resulting in very low precision.

Table 4.2 *Zero-shot SLM vs. T-BERT with Confusion Matrix Statistics*

Dataset	Model	Precision (%)	Recall (%)	TP	FP	TN	FN
Test Project	T-BERT (Baseline)	75.00	100.00	9	3	33	0
	Code Llama (Prompt 1)	40.91	100.00	9	13	23	0
	Code Llama (Prompt 2)	40.91	100.00	9	13	23	0
EV-Components (GUI)	T-BERT (Baseline)	43.75	100.00	7	9	425	0
	Code Llama (Prompt 1)	3.32	100.00	7	204	230	0
	Code Llama (Prompt 2)	4.61	100.00	7	145	289	0
Liquid AI-Supervisor	T-BERT (Baseline)	62.50	100.00	10	6	4791	0
	Code Llama (Prompt 1)	0.38	100.00	10	2648	2149	0
	Code Llama (Prompt 2)	0.53	100.00	10	1894	2903	0

Key observation On all datasets, the SLM is strongly biased toward predicting links as positive, leading to perfect recall but extremely low precision on the larger projects. Prompt 2 reduces false positives compared to Prompt 1, but precision still remains very low.

Interpretation The results show that the SLM consistently predicts a large number of issue-commit pairs as linked, which leads to perfect recall but very low precision, particularly on the larger repositories. This behavior suggests that, under the evaluated zero-shot prompting setup, the model tends to over-associate commits with issue descriptions when diff content and commit messages share related vocabulary or functionality cues.

In practical settings, such behavior would impose a high verification burden on developers because many predicted links would be incorrect. Compared to T-BERT, which produces substantially fewer false positives under the same datasets, the SLM predictions are considerably less selective.

These observations indicate that the zero-shot SLM configuration used in this study lacks mechanisms to sufficiently constrain link predictions. Later in RQ3, we

investigate whether incorporating repository context through retrieval-augmented generation can help mitigate this behavior.

4.3 RQ3: Impact of RAG-Based Repository Grounding on Traceability

This section evaluates whether repository grounding via Retrieval-Augmented Generation improves SLM-based traceability decisions. As a sub-question (RQ3.1), we vary the retrieval depth $K \in \{1, 5, 10, 15, 20, 30\}$ and report (i) the number of true positives recovered and (ii) the total number of predicted links as a proxy for verification burden. In addition, we explicitly report false positives (FP) as an interpretable measure of incorrect candidate links; in our setup, $FP = (\text{total predictions}) - TP$.

4.3.1 Effect of Retrieval Depth (K) on Link Recovery

Figure 4.1 illustrates how the total prediction volume, recovered true positives, and false positives change as K increases, on the evaluation datasets.

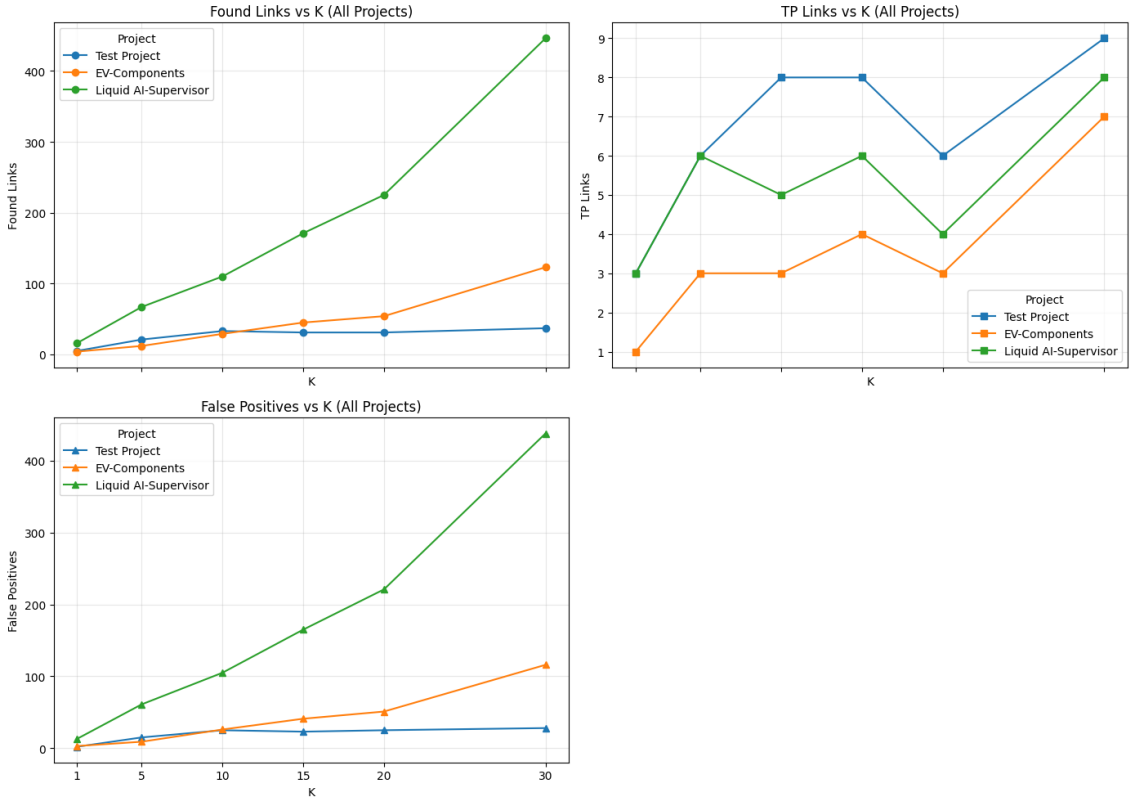


Figure 4.1 RAG performance as a function of retrieval depth (K) across all evaluation projects, showing total predictions (top-left), true positives recovered (top-right), and false positives (bottom-left).

Key observation Increasing K generally increases the number of recovered true positives across projects. For example, in the Test Project dataset, true positives increase from 3 at $K = 1$ to 9 at $K = 30$, reaching full recovery of the expert-labeled links.

At the same time, the number of false positives also increases with K (Figure 4.1). This growth is modest for the Test Project but becomes substantial on the larger repositories. For instance, Liquid AI-Supervisor grows from 13 false positives at $K = 1$ (16 predictions with 3 TP) to 438 false positives at $K = 30$ (446 predictions with 8 TP), indicating that much of the additional prediction volume at higher K consists of incorrect links.

4.3.2 Prediction Volume and Precision Trade-Off

The Table 4.3 reports true positives recovered and total predictions for each dataset and retrieval depth.

Table 4.3 RAG results across retrieval depths K shown as $TP/(total\ predictions)$, with Precision and Recall at each K

Project	Total Valid Links	K	TP/(Tot)	Precision (%)	Recall (%)
Test Project	9	1	3/(5)	60.00	33.33
		5	6/(21)	28.57	66.67
		10	8/(33)	24.24	88.89
		15	8/(31)	25.81	88.89
		20	6/(31)	19.35	66.67
		30	9/(37)	24.32	100.00
EV-Components	7	1	1/(4)	25.00	14.29
		5	3/(12)	25.00	42.86
		10	3/(29)	10.34	42.86
		15	4/(45)	8.89	57.14
		20	3/(54)	5.56	42.86
		30	7/(123)	5.69	100.00
Liquid AI-Supervisor	10	1	3/(16)	18.75	30.00
		5	6/(67)	8.96	60.00
		10	5/(110)	4.55	50.00
		15	6/(171)	3.51	60.00
		20	4/(225)	1.78	40.00
		30	8/(446)	1.79	80.00

Key observation Higher retrieval depth increases recovered true positives but also increases the number of predicted links substantially (e.g., Liquid AI-Supervisor

grows from 16 to 446 predictions from $K = 1$ to $K = 30$). This indicates a clear trade-off between recall gains and verification burden.

Interpretation RAG improves recovery of true links as K increases, but the benefit comes with a rapid growth in predicted links, which reduces precision at higher retrieval depths. The FP trend in Figure 4.1 makes this trade-off explicit: higher retrieval depth injects more retrieved evidence into the prompt and surfaces more candidates, but many of the additional candidates are incorrect links. As a result, recall tends to improve with K , whereas the verification burden increases sharply due to accumulating false positives (especially on larger projects). This suggests that repository grounding helps the SLM recover relevant links, but additional mechanisms are needed to control false positives (e.g., tighter retrieval, improved prompting constraints, or post-filtering).

4.3.3 Comparison with T-BERT Baseline

To contextualize the RAG+SLM results, we compare them against the T-BERT baseline reported in RQ2 on the same datasets and expert labels. The comparison focuses on two quantities that are directly observable in our evaluation setup: (i) the number of recovered true positives (TP) and (ii) the total number of predicted links, which serves as a proxy for verification burden (Tables 4.2 and Table 4.3).

Across all three projects, T-BERT produces a comparatively small number of predicted links in absolute terms (12 predictions in Test Project, 16 in EV-Components, and 16 in Liquid AI-Supervisor), while still recovering all valid links in these datasets. In contrast, the RAG+SLM pipeline produces a number of predictions that increase with retrieval depth K , reflecting the fact that more retrieved evidence is injected into the prompt and more candidate links are surfaced.

In terms of recovered true positives, RAG+SLM generally improves with larger K . For instance, in the Test Project, TP increases from 3 at $K = 1$ to 9 at $K = 30$. A similar pattern is observed in EV-Components (from 1 to 7) and Liquid AI-Supervisor (from 3 to 8), although the gains are not strictly monotonic across intermediate K values. At the same time, the total number of predicted links grows substantially as K increases.

Overall, this comparison shows that RAG+SLM provides a controllable mechanism that increases recovered true links while also increasing prediction volume, whereas T-BERT exhibits a lower prediction volume under the same datasets. The implications of these differing behaviors for precision-recall trade-offs and practical verification effort are discussed in Chapter 5.

5 Discussion

This chapter explains what the results in Chapter 4 mean and how they relate to the research questions and prior work on automated Traceability Link Recovery (TLR). The aim of this thesis was not to propose a new traceability system, but to test how practical design choices affect performance when we use real repositories and realistic constraints.

The discussion follows the three research questions (RQ1–RQ3). It then outlines recommendations and future work for building traceability tools, and discusses threats to validity.

5.1 RQ1: Impact of Input Restructuring on T-BERT Performance

RQ1 examined how different commit diff input restructuring strategies affect T-BERT performance in traceability link recovery. Two alternative diff representations were evaluated against the baseline preprocessing format used in prior work: (i) an ordered-diff representation that preserves the sequential structure of code changes, and (ii) a variant that retains only added lines from the diff. Across the evaluated repositories, neither restructuring strategy improved performance over the baseline, and both produced results that were similar to or slightly below the baseline.

These results suggest that changing the surface representation of commit diffs alone is not sufficient to improve T-BERT in the current setup. Although the tested strategies modify how code changes are presented to the model, the overall traceability signal available to T-BERT may remain largely unchanged, especially when key lexical cues are preserved across formats.

One likely explanation is the input length limitation of T-BERT. The model processes a maximum of 512 tokens, so long issue–commit pairs are truncated before inference. Under this constraint, improvements introduced by a more structured diff representation may not be consistently visible to the model, because substantial parts of the issue and/or commit content can be removed. This is particularly relevant for the ordered-diff variant, where preserving sequential information is only useful if enough of that sequence remains within the visible context window.

To further examine this explanation, a token length distribution analysis was performed for the baseline, Version 1, and Version 2 inputs. Figures 5.1 and 5.2 compare the proportion of issue–commit pairs that fall within and exceed the 512-token limit across projects. The baseline representation shows the lowest rate of

input truncation, with only 12.0%–20.0% of pairs exceeding the limit across repositories. In contrast, Version 1 produces substantially more pairs which gets truncated, ranging from 41.6% to 66.2%, with `keras` again showing the highest proportion. Version 2 reduces this proportion considerably to 27.4%–35.2%, indicating that the added-lines-only representation is more length-efficient than Version 1, though still less compact than the baseline. Overall, the results of this analysis confirm that truncation is most severe for Version 1, less pronounced for Version 2, and comparatively limited in the baseline input format. This strengthens the interpretation that input length is an important constraint in the current setup. At the same time, because the baseline already retains the largest proportion of issue–commit pairs within the 512-token window and still outperforms the two alternative restructuring strategies.

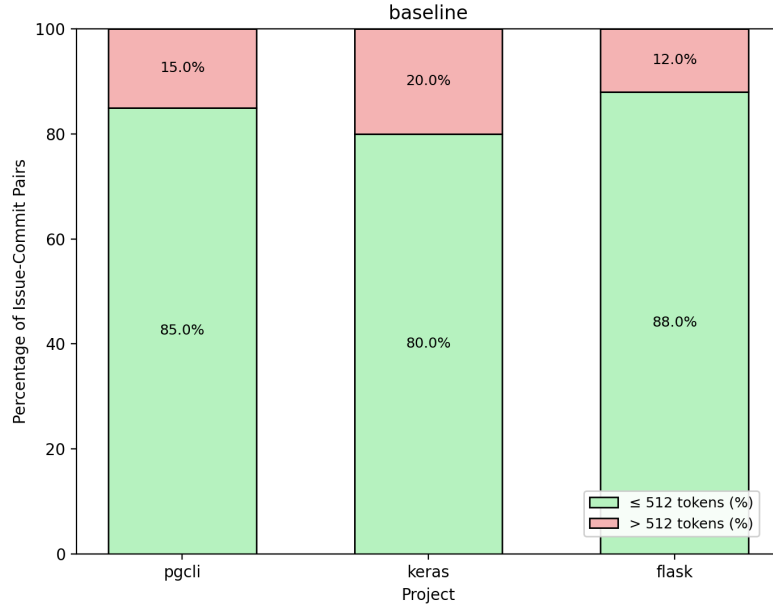


Figure 5.1 Token Length Distribution by Project for Baseline T-BERT Inputs

T-BERT’s performance relies heavily on strong lexical and semantic cues already present in the baseline representation, as shown by [32]. In issue–commit linking tasks, identifiers, API names, file paths, and other project-specific terms carry much of the traceability signal. Because these cues are still present in both restructuring strategies, changes such as line reordering or git diff markers provide only limited additional benefit. Another reason is that the model was not originally trained on git diff style content during intermediate training. Instead, T-BERT learned from function documentation and implementation pairs [23]. Compared with that training data, commit diffs contain a noisier structure, including git diff markers and other change-related symbols. This makes the input harder for the model to

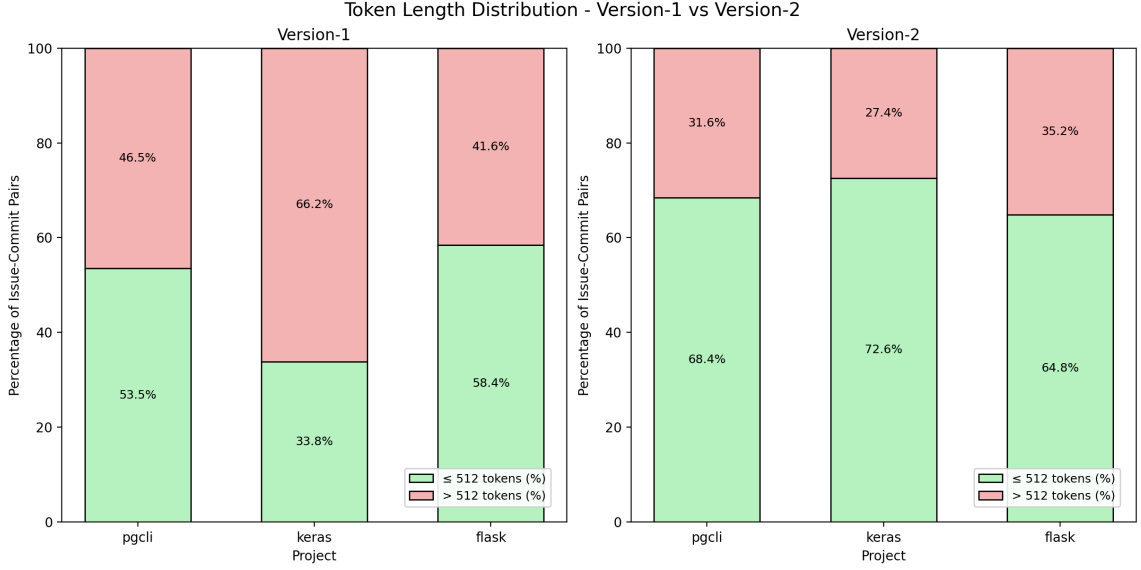


Figure 5.2 Token Length Distribution by Project for Version 1 and Version 2 Inputs

interpret. When the diff format is changed further through restructuring, the input becomes even less similar to the type of data the model learned from earlier, which can make performance worse.

Overall, the RQ1 findings indicate that the tested input restructuring strategies had limited impact on T-BERT performance in this study. This does not imply that commit diff structure is irrelevant for traceability link recovery. Rather, it suggests that representation-level changes alone may be insufficient under the current model and preprocessing constraints. More substantial gains may require combining improved input design with better context selection, truncation-aware preprocessing, or architectures that can utilize longer contexts more effectively.

5.2 RQ2: The SLM Behavior and the Over-Linking Problem

RQ2 examined how a zero-shot Small Language Model (SLM) performs on traceability link recovery compared to the T-BERT baseline. The results show a clear pattern, recall was very high, but precision dropped strongly as the repository became larger. In practice, the model predicted too many links.

In open-source repositories, true links are rare compared to all possible issue-commit pairs. In a zero-shot prompt, the model is not explicitly trained to account for this imbalance [38]. As a result, it tends to treat local similarity (shared terms and similar themes) as sufficient evidence for a traceability link, which leads to many false positives.

Compute-time implications and method-level comparison Beyond predictive quality, RQ2 highlights an important difference between prompting-based and

encoder-based approaches to traceability link recovery. The zero-shot SLM setup requires running one model inference per Issue–Commit pair. For a repository with $|I|$ issues and $|C|$ commits, this results in $|I| \times |C|$ model calls, which scales quadratically and becomes computationally expensive even for moderately sized projects.

At a higher level, this reflects a fundamental distinction between prompting-based and representation-based methods. Encoder models such as T-BERT learn fixed vector representations of artifacts and can compare them efficiently using similarity measures, enabling batching and approximate nearest neighbor search [36, 23]. In contrast, prompting-based SLMs rely on autoregressive generation for each decision, making them significantly more expensive per pair and less amenable to large-scale candidate ranking [4, 38].

This difference implies that while SLMs may offer flexibility and strong zero-shot capabilities, encoder-based approaches remain more practical for large-scale traceability tasks. Consequently, the computational limitations observed in RQ2 are not only an implementation issue, but a broader characteristic of prompting-based methods. This motivates the repository-grounding strategy explored in RQ3, which aims to reduce the number of required model calls by narrowing the candidate space.

Prompt length and truncation Long diffs still increase lexical overlap with issue text, which can encourage the model to interpret weak similarities (generic terms or reused identifiers) as evidence for a link. However, prompt truncation is not the primary factor in our evaluation setting. As shown in Figure 5.3, most Issue–Commit prompts fits within the context window, truncation occurs in 0% of the Test Project pairs, 4.7% of the *wasmiot-supervisor* pairs, and 16.7% of the GUI pairs. These proportions are reported for prompt template 2, as it contains only a few additional tokens compared with template 1. This indicates that the SLM’s over-linking behavior is present even when the full prompt fits into the context window.

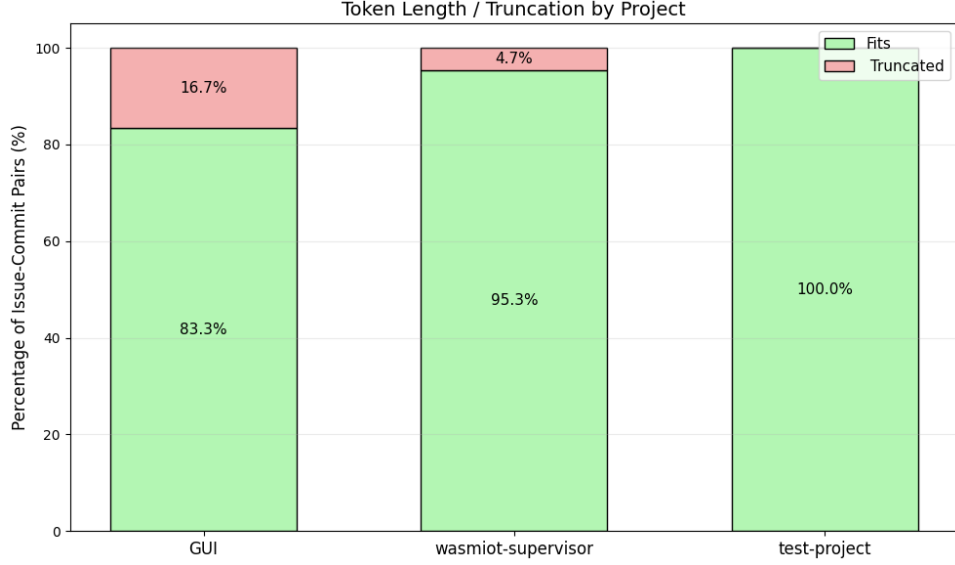


Figure 5.3 Proportion of SLM prompt-2 that exceed the model context window and are truncated, reported per project

Effect of incomplete labeling on measured precision. Measured precision is also affected by the reference labeling strategy as mentioned in Section 3.5.1. For evaluation, pairs without expert labels are treated as **False** by default, which is a conservative assumption but can introduce label noise. In repositories where only a small subset of true issue–commit links are labeled, some predictions counted as false positives may correspond to valid but undocumented links. This limitation can disproportionately penalize systems that predict more positives (such as the zero-shot SLM) and is addressed in Section 5.5.

Prompt changes made the model slightly more conservative, but not enough to make it competitive with the supervised baseline. This suggests that prompt engineering alone is not a reliable solution when precision matters [38].

From a deployment perspective, this is important, in most software engineering settings, high precision is usually more valuable than perfect recall. Too many false links increase review effort and reduce developer trust. Together with the compute-time cost of $|I| \times |C|$ pairwise inference, these results indicate that zero-shot SLMs require additional grounding, calibration, or candidate narrowing before they can be used as practical link predictors in larger repositories.

5.3 RQ3: RAG Improves Recall but Increases Verification Effort

RQ3 asked whether Retrieval-Augmented Generation (RAG)-based repository grounding improves SLM-based traceability, with retrieval depth (K) treated as a secondary

factor (RQ3.1). The results show that higher K generally increased the chance of retrieving relevant context, which helped the model find true links.

However, higher K also caused the model to output far more links overall, especially in larger repositories. In those cases, many retrieved chunks were not truly relevant, but they still influenced the model. The SLM often treated partially related context as support for a link, which increased false positives.

This creates a clear trade-off:

- **More retrieval** $\rightarrow$ more chances to include the right evidence $\rightarrow$ higher recall.
- **More retrieval** $\rightarrow$ more noisy context $\rightarrow$ lower precision and more links to verify.

The key takeaway is that RAG helps with repository grounding, but grounding alone does not solve the SLM’s tendency to over-associate. Practical RAG pipelines need additional controls, such as stricter retrieval filters, re-ranking, better chunking strategies, or a mechanism for the model to abstain when evidence is weak.

5.4 Recommendations and Future Work for Building Traceability Tools

Overall, the results suggest that practical traceability tools should be designed around the constraints of the underlying models and the realities of developer workflows, especially limited context windows, strong class imbalance (true links are rare), and the cost of manual verification. The main implication is that improving usability and precision often matters more than maximizing raw recall, because false links directly translate into review effort.

First, for encoder-based approaches such as T-BERT, the findings from RQ1 indicate that simply reformatting commit diffs (e.g., preserving ordered structure or keeping only added lines) is unlikely to improve performance on its own. Under fixed input limits (e.g., 512 tokens), important evidence is frequently truncated, which can outweigh any benefit from a more structured representation. For software engineers, this points to a more actionable strategy: prioritize selecting and retaining the most informative parts of the issue and commit content, rather than spending effort on changing the surface structure of the diff while keeping largely the same information.

Second, as shown in RQ2, the zero-shot SLMs can achieve high recall but tend to over-link, particularly as the repository size grows. This over-linking reduces precision, lowering developer trust in practical settings. It also has compute-time implications: pairwise prompting requires one inference per Issue–Commit pair, scaling as $|I| \times |C|$, which becomes expensive even for moderately sized projects.

Consequently, zero-shot TLR using SLM should not be used as a stand-alone mechanism.

Third, RQ3 indicates that repository grounding via RAG can improve recall, but it also increases the number of predicted links and therefore the likely verification burden when retrieval depth K is increased. This trade-off is reflected in the empirical results in Table 4.3. For practical system design, K should be treated as a workflow parameter: smaller values are preferable when developers need concise, high-confidence candidates, whereas larger values may be justified when maximizing recovery is more important and additional review effort is acceptable. More broadly, grounding helps, but it does not fully address the model’s tendency to treat weak or partially related evidence as sufficient support for a link.

Based on these implications, the most promising future work is (i) truncation-aware artifact construction that keeps the most relevant diff and issue content within the available token budget, (ii) calibration for SLMs (e.g., fine-tuning, thresholding, or context engineering) to reduce over-linking when evidence is weak, and (iii) stronger RAG pipelines that reduce retrieval noise through better chunking, filtering, and re-ranking so that additional retrieval does not increase false positives.

5.5 Threats to Validity

This section discusses potential threats to internal, external, and construct validity that may influence the interpretation of the results.

A general threat to validity in this study is the trade-off between breadth and depth. The study evaluates three different traceability approaches, but each approach was examined only within a limited configuration space rather than through extensive optimization or ablation. Consequently, the results should be interpreted as comparative observations across selected implementations, not as definitive evidence of the full potential of each approach.

Internal validity concerns whether the observed effects can be attributed to the experimental manipulations rather than uncontrolled factors. In RQ1, only two ordered-diff restructuring strategies were evaluated in addition to the baseline representation. Other possible input constructions were not explored. Therefore, the conclusion that ordered restructuring does not improve T-BERT performance applies only to the specific variants implemented in this study.

For RQ2, the SLM was evaluated strictly in a zero-shot setting without fine-tuning, few-shot examples, threshold calibration, or post-processing. Only two prompt templates were evaluated. Prompt engineering can significantly influence the behavior of these models, and alternative prompt designs may lead to different precision–recall trade-offs. The absence of systematic prompt engineering or template variation means that the reported zero-shot performance reflects the specific

prompt configurations used in this study rather than the full capability of the model under optimized prompting.

For RQ3, retrieval depth (K) was varied systematically, but other RAG design dimensions such as embedding models, chunk sizes, re-ranking strategies, filtering mechanisms, or prompt-context integration strategies were not controlled or compared. As a result, the observed precision–recall trade-off reflects the specific RAG configuration implemented rather than the full design space of retrieval-augmented systems.

External validity concerns the generalizability of the findings beyond the evaluated repositories. The study includes multiple repositories of different sizes and domains, but they do not represent the full diversity of software engineering projects. Repository characteristics such as programming language, commit practices, documentation quality, and linking conventions may affect traceability performance. Therefore, the results should not be generalized to all repositories without further empirical validation on additional projects.

Construct validity concerns whether the chosen evaluation measures accurately capture traceability performance. For RQ1, MAP@3 and MRR emphasize early ranking quality but do not reflect the full ranking distribution. For RQ2 and RQ3, precision and recall are sensitive to class imbalance, and the evaluated datasets contain relatively few positive links compared to total candidate pairs. In such settings, small changes in false positive counts can substantially affect precision values.

Ground truth labels may also be incomplete or inconsistent. Some predicted false positives may correspond to valid but undocumented links, while some labeled links may reflect repository conventions rather than strict semantic alignment. No explicit mitigation strategies were applied to address these threats. Therefore, the results should be interpreted as empirical observations under defined experimental conditions rather than definitive proof of the superiority of one method over another.

6 Conclusion

This thesis investigated how the precision of automated Traceability Link Recovery (TLR) can be improved in real-world GitHub repositories, building directly on the findings of Parveen’s Master’s thesis [30]. The work was motivated by the persistent challenge of maintaining traceability links between issues and commits in modern software projects. As discussed in Chapter 1, traceability is essential for activities such as maintenance, impact analysis, and knowledge transfer, yet it is rarely maintained systematically in collaborative development environments. The central goal of this research was therefore to examine whether the precision limitations observed in T-BERT under realistic repository conditions could be reduced through alternative input design and, when necessary, through broader modeling strategies that remain practical for real-world use.

The research followed an iterative experimental process starting from an established baseline and gradually exploring alternative modeling directions. First, we examined whether restructuring commit diffs could improve the performance of the T-BERT traceability model. The motivation behind this experiment was that commit diffs naturally contain structured information about code changes, and preserving their sequential order or emphasizing added lines could potentially strengthen the signal available to the model. However, the results showed that neither restructuring strategy improved the baseline performance. The ordered-diff variants produced results that were similar to or slightly below the original preprocessing approach. These findings suggest that modifying the surface representation of commit diffs alone is insufficient to substantially improve T-BERT under the current experimental constraints. In particular, the fixed input length and truncation effects likely limit how much of the structured diff information the model can actually process.

After observing the limitations of the T-BERT architecture, the research direction shifted toward evaluating whether a Small Language Model (SLM) could provide a more direct decision-based approach to linking issues and commits. Unlike the T-BERT setup, the SLM formulation allows the model to directly assess whether a given issue–commit pair represents a valid traceability link. The results demonstrated that the SLM approach can recover relevant links, but it also tended to produce a large number of predictions, leading to increased false positives. This behavior indicates that while generative language models can capture semantic relationships between development artifacts, they may lack sufficient grounding when applied to large sets of candidate pairs. In practice, this can increase the verification effort required from developers, which is an important consideration for real-world

adoption.

To address this limitation, the final stage of the study introduced Retrieval-Augmented Generation (RAG) to provide additional repository context and reduce the need for exhaustive pairwise comparisons. Instead of evaluating every possible issue–commit pair, the RAG pipeline retrieves a limited set of semantically related commit fragments for each issue and incorporates this contextual evidence into the prompt, thereby grounding the SLM’s decision in repository-local evidence. The results show that RAG can increase the recovery of true links, but that this benefit depends on retrieval design. In particular, increasing retrieval depth K generally improves the number of recovered true links, while also increasing the number of predicted links overall, highlighting a clear trade-off between recall gains and verification effort. Compared with the earlier experiments, the RAG-based approach offers a controllable mechanism for balancing these factors, making it a promising direction for practical traceability systems.

Taken together, the findings of this thesis illustrate that improving automated traceability recovery is not solely a matter of choosing a more powerful model. Architectural constraints, input representations, and retrieval strategies all influence how effectively traceability signals can be extracted from repository artifacts. The experiments demonstrate that simple preprocessing changes may have limited impact when model constraints remain unchanged, while retrieval-based approaches can provide a more flexible mechanism for incorporating repository context. These observations contribute to a better understanding of how modern language models behave in traceability tasks under realistic development conditions.

Several directions for future work emerge from this study. One promising area is truncation-aware input construction, where the most informative sections of commit diffs are selected before being passed to the model. This could help mitigate the input length limitations observed in the T-BERT experiments. Another direction is improving calibration and fine-tuning strategies for small language models so that they produce fewer false positives when reasoning about artifact relationships. Finally, more advanced retrieval pipelines could further improve the effectiveness of RAG-based traceability, for example, by incorporating improved chunking strategies, filtering mechanisms, or re-ranking methods to reduce irrelevant context before generation.

In summary, this thesis explored multiple approaches to automated traceability link recovery and highlighted the practical trade-offs between encoder-based models, generative language models, and retrieval-augmented methods. While no single approach fully resolves the challenges of traceability recovery, the results demonstrate that retrieval-grounded language models offer a promising path forward. Continued research in this direction, particularly in improving context selection and model

calibration, can help move automated traceability closer to practical deployment in real software development environments.

References

- [1] A. Abadi, M. Nisenson, and Y. Simionovici. “A traceability technique for specifications”. In: *The 16th IEEE International Conference on Program Comprehension*. IEEE, 2008, pp. 103–112. DOI: 10.1109/ICPC.2008.30.
- [2] G. Antoniol, G. Canfora, G. Casazza, A. De Lucia, and E. Merlo. “Recovering traceability links between code and documentation”. In: *IEEE Transactions on Software Engineering* 28.10 (2002), pp. 970–983. DOI: 10.1109/TSE.2002.1041053.
- [3] M. Borg, P. Runeson, and A. Ardö. “Recovering from a decade: A systematic mapping of information retrieval approaches to software traceability”. In: *Empirical Software Engineering* 19.6 (2014), pp. 1565–1616. DOI: 10.1007/s10664-013-9255-y.
- [4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.
- [5] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. “Learning phrase representations using RNN encoder-decoder for statistical machine translation”. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*. 2014, pp. 1724–1734. DOI: 10.3115/v1/D14-1179.
- [6] J. Cleland-Huang, O. Gotel, and A. Zisman. *Software and Systems Traceability*. Springer, 2012. ISBN: 978-1-4471-2238-8. DOI: 10.1007/978-1-4471-2239-5.
- [7] F. Dalpiaz and N. Niu. “Requirements Engineering in the Days of Artificial Intelligence”. eng. In: *IEEE software* 37.4 (2020), pp. 7–10. ISSN: 0740-7459.
- [8] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer, and R. Harshman. “Indexing by latent semantic analysis”. In: *Journal of the American Society for Information Science* 41.6 (1990), pp. 391–407. DOI: 10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9.
- [9] Y. Deng, B. Wang, Q. Zhu, J. Liu, J. Kuang, and X. Li. “MTLink: Adaptive multi-task learning based pre-trained language model for traceability link recovery between issues and commits”. eng. In: *Journal of King Saud University. Computer and information sciences* 36.2 (2024), pp. 101958–. ISSN: 1319-1578.

- [10] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova. “BERT: Pre-training of deep bidirectional transformers for language understanding”. eng. In: *NAACL HLT 2019 - 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference*. Vol. 1. 2019, pp. 4171–4186. ISBN: 978-1-950737-13-0.
- [11] B. Dit, M. Reville, M. Gethers, and D. Poshyvanyk. “Feature location in source code: A taxonomy and survey”. In: *Journal of Software: Evolution and Process* 25.1 (2013), pp. 53–95. DOI: 10.1002/smr.567.
- [12] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou. “CodeBERT: A Pre-Trained Model for Programming and Natural Languages”. eng. In: *arXiv.org* (2020). Place: Ithaca Publisher: Cornell University Library, arXiv.org. ISSN: 2331-8422.
- [13] *GitHub REST API documentation*. en. URL: https://docs-internal.github.com/_next/data/wxEpRz9dG8eDiCZg0A7i8/en/free-pro-team%40latest/rest.json?apiVersion=2022-11-28&versionId=free-pro-team%40latest&productId=rest.
- [14] *GitPython Documentation — GitPython 3.1.45 documentation*. URL: <https://gitpython.readthedocs.io/en/stable/>.
- [15] O. C. Gotel and A. C. Finkelstein. “An analysis of the requirements traceability problem”. In: *Proceedings of the First International Conference on Requirements Engineering*. IEEE, 1994, pp. 94–101. DOI: 10.1109/ICRE.1994.292398.
- [16] J. Guo, J. Cheng, and J. Cleland-Huang. “Semantically enhanced software traceability using deep learning techniques”. In: *IEEE/ACM 39th International Conference on Software Engineering*. IEEE, 2017, pp. 3–14. DOI: 10.1109/ICSE.2017.9.
- [17] T. Hey, D. Fuchß, J. Keim, A. Koziolk, A. Susi, and A. Hess. “Requirements Traceability Link Recovery via Retrieval-Augmented Generation”. eng. In: *Requirements Engineering: Foundation for Software Quality*. Vol. 15588. Cham: Springer Nature Switzerland, 2025, pp. 381–397. ISBN: 978-3-031-88530-3.
- [18] S. Hochreiter and J. Schmidhuber. “Long short-term memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [19] H. Hu, S. Yu, P. Chen, and E. M. Ponti. “Fine-Tuning Large Language Models with Sequential Instructions”. eng. In: *Proceedings of the 2025 Annual Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies: Long Papers, NAACL-HLT 2025*. Vol. 1. 2025, pp. 5589–5610. ISBN: 979-8-89176-189-6.

- [20] H. Husain, W. Ho-Hsiang, T. Gazit, M. Allamanis, and M. Brockschmidt. “CodeSearchNet Challenge: Evaluating the State of Semantic Code Search”. eng. In: *arXiv.org* (2020). Place: Ithaca Publisher: Cornell University Library, arXiv.org. ISSN: 2331-8422.
- [21] J. Lan, L. Gong, J. Zhang, and H. Zhang. “BTLink : automatic link recovery between issues and commits based on pre-trained BERT model”. eng. In: *Empirical software engineering : an international journal* 28.4 (2023). Place: New York, pp. 103–. ISSN: 1382-3256.
- [22] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. “Retrieval-augmented generation for knowledge-intensive NLP tasks”. eng. In: *34th Conference on Neural Information Processing Systems, NeurIPS 2020*. Vol. 2020-. ISSN: 1049-5258. 2020.
- [23] J. Lin, Y. Liu, Q. Zeng, M. Jiang, and J. Cleland-Huang. “Traceability Transformed: Generating more Accurate Links with Pre-Trained BERT Models”. eng. In: *Proceedings / International Conference on Software Engineering*. ISSN: 1558-1225. Piscataway, NJ, USA: IEEE Press, 2021, pp. 324–335. ISBN: 1-4503-9085-4.
- [24] P. Mader and A. Egyed. “An empirical study of the documentation essential to software maintenance”. In: *International Symposium on Empirical Software Engineering and Measurement*. IEEE, 2012, pp. 53–60. DOI: 10.1145/2372251.2372262.
- [25] C. D. Manning, P. Raghavan, and H. Schütze. *Introduction to information retrieval*. eng. Publication Title: Introduction to information retrieval. Cambridge: Cambridge University Press, 2008. ISBN: 1-139-63743-6.
- [26] A. Marcus and J. I. Maletic. “Recovering documentation-to-source-code traceability links using latent semantic indexing”. In: *25th International Conference on Software Engineering*. IEEE, 2003, pp. 125–135. DOI: 10.1109/ICSE.2003.1201194.
- [27] A. Mehraj, Z. Zhang, and K. Systä. “A tertiary study on AI for requirements engineering”. In: *International Working Conference on Requirements Engineering: Foundation for Software Quality*. 2024, pp. 159–177.
- [28] T. Mikolov, K. Chen, G. Corrado, and J. Dean. “Efficient estimation of word representations in vector space”. In: *arXiv preprint arXiv:1301.3781* (2013).

- [29] C. Mills, E. Navarro, and S. Haiduc. “Tracing with less data: Active learning for classification-based traceability link recovery”. In: *IEEE International Conference on Software Maintenance and Evolution*. IEEE, 2018, pp. 453–463. DOI: 10.1109/ICSME.2018.00055.
- [30] R. Parveen. “Investigating T-BERT for Automated Issue–Commit Link Recovery”. eng. MA thesis. Tampere University, 2025.
- [31] R. Parveen, A. Mehraj, Z. Zhang, K. Systä, and T. Kilamo. “Towards automated recovery of links between code commits and requirements—initial results”. In: *International Conference on Product-Focused Software Process Improvement*. 2024, pp. 386–394.
- [32] R. Parveen, Z. Zhang, K. Systä, T. Kilamo, and A. Mehraj. “Towards Effective Automation of Issue–Commit Link Recovery: An Empirical Investigation”. In: *International Conference on Product-Focused Software Process Improvement*. 2025, pp. 237–252.
- [33] J. Pennington, R. Socher, and C. D. Manning. “GloVe: Global vectors for word representation”. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2014, pp. 1532–1543. DOI: 10.3115/v1/D14-1162.
- [34] B. Ramesh, C. Stubbs, T. Powers, and M. Edwards. “Requirements traceability: Theory and practice”. eng. In: *Annals of software engineering* 3.1 (1997), pp. 397–415. ISSN: 1022-7091.
- [35] M. Rath and P. Mäder. “IlmSeven: A Traceability-Supporting Software Project Process Involving Seven Bachelor’s Students”. In: *Proceedings of the 39th International Conference on Software Engineering Companion*. IEEE, 2017, pp. 217–218. DOI: 10.1109/ICSE-C.2017.108.
- [36] N. Reimers and I. Gurevych. “Sentence-BERT: Sentence embeddings using Siamese BERT-networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410.
- [37] D. Riehle, C. Kolassa, and M. A. Salim. “Developer belief vs. reality: The case of the commit size distribution”. In: *arXiv preprint arXiv:1408.4644* (2014).
- [38] A. D. Rodriguez, K. R. Dearstyne, and J. Cleland-Huang. “Prompts matter: Insights and strategies for prompt engineering in automated software traceability”. In: *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*. IEEE. 2023, pp. 455–464.

- [39] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve. “Code Llama: Open Foundation Models for Code”. eng. In: *arXiv.org* (2024). Place: Ithaca Publisher: Cornell University Library, arXiv.org. ISSN: 2331-8422.
- [40] G. Salton, A. Wong, and C.-S. Yang. “A vector space model for automatic indexing”. In: *Communications of the ACM* 18.11 (1975), pp. 613–620. DOI: 10.1145/361219.361220.
- [41] R. Torkar, T. Gorschek, R. Feldt, M. Svanberg, U. A. Raja, and K. Kamran. “Requirements Traceability: A Systematic Review and Industry Case Study”. eng. In: *International journal of software engineering and knowledge engineering* 22.3 (2012). Place: SINGAPORE, pp. 385–433. ISSN: 0218-1940.
- [42] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention is all you need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 5998–6008.
- [43] H. S. Vera, S. Dua, B. Zhang, D. Salz, R. Mullins, S. R. Panyam, S. Smoot, I. Naim, J. Zou, F. Chen, D. Cer, A. Lisak, M. Choi, L. Gonzalez, O. Sanseviero, G. Cameron, I. Ballantyne, K. Black, K. Chen, W. Wang, Z. Li, G. Martins, J. Lee, M. Sherwood, J. Ji, R. Wu, J. Zheng, J. Singh, A. Sharma, D. Sreepathihalli, A. Jain, A. Elarabawy, A. J. Co, A. Doumanoglou, B. Samari, B. Hora, B. Potetz, D. Kim, E. Alfonseca, F. Moiseev, F. Han, F. P. Gomez, G. H. Ábrego, H. Zhang, H. Hui, J. Han, K. Gill, K. Chen, K. Chen, M. Shanbhogue, M. Boratko, P. Suganthan, S. M. K. Duddu, S. Mariserla, S. Ariaifar, S. Zhang, S. Zhang, S. Baumgartner, S. Goenka, S. Qiu, T. Dabral, T. Walker, V. Rao, W. Khawaja, W. Zhou, X. Ren, Y. Xia, Y. Chen, Y.-T. Chen, Z. Dong, Z. Ding, F. Visin, G. Liu, J. Zhang, K. Kenealy, M. Casbon, R. Kumar, T. Mesnard, Z. Gleicher, C. Brick, O. Lacombe, A. Roberts, Q. Yin, Y. Sung, R. Hoffmann, T. Warkentin, A. Joulin, T. Duerig, and M. Seyedhosseini. *EmbeddingGemma: Powerful and Lightweight Text Representations*. arXiv:2509.20354 [cs]. Nov. 2025. DOI: 10.48550/arXiv.2509.20354. URL: <http://arxiv.org/abs/2509.20354>.
- [44] F. Wang, M. Lin, Y. Ma, H. Liu, Q. He, X. Tang, J. Tang, J. Pei, and S. Wang. “A survey on small language models in the era of large language models: Architecture, capabilities, and trustworthiness”. In: *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 2025, pp. 6173–6183.

- [45] X. Ye, R. Bunescu, and C. Liu. “Mapping bug reports to relevant files: A ranking model, a fine-grained benchmark, and feature evaluation”. In: *IEEE Transactions on Software Engineering* 42.8 (2016), pp. 675–699. DOI: 10.1109/TSE.2016.2519032.
- [46] Q. Zhang, C. Fang, Y. Xie, Y. Zhang, Y. Yang, W. Sun, S. Yu, and Z. Chen. “A survey on large language models for software engineering”. In: *arXiv preprint arXiv:2312.15223* (2023).
- [47] J. Zhu, G. Xiao, Z. Zheng, and Y. Sui. “Deep semi-supervised learning for recovering traceability links between issues and commits”. eng. In: *The Journal of systems and software* 216 (2024), pp. 112109–. ISSN: 0164-1212.

APPENDIX A. Prompt Templates

RQ2 - Prompt Template 1

[INST] You are a code analysis assistant. Your task is to determine if a commit's code changes are related to a given issue description. Analyze the following information and respond with only "True" if the code changes address or relate to the issue, or "False" if they are unrelated.

Issue Description:

{issue_description}

Commit Summary:

{commit_summary}

Code Changes:

{code_diff}

Instructions:

- Consider if the code changes directly address the problem described in the issue
- Look for keywords, function names, or logic that relates to the issue
- Consider if the commit summary indicates intent to fix the issue
- Respond with only "True" or "False" - no explanation needed

Answer:

[/INST]

RQ2 - Prompt Template 2

[INST] You are a code analysis assistant. Your task is to determine if a commit's code changes are related to a given issue description. Analyze the following information and respond with whether the code changes are related to the issue, along with your confidence level.

Issue Description:

{issue_description}

Commit Summary:

{commit_summary}

Code Changes:

{code_diff}

Instructions:

- Consider if the code changes directly address the problem described in the issue
- Look for keywords, function names, or logic that relates to the issue
- Consider if the commit summary indicates intent to fix the issue
- Respond with your answer and confidence score in this exact format:
"ANSWER: [True/False], CONFIDENCE: [0-100]"
- Confidence score should be 0-100, where 100 means completely certain

Answer:

[/INST]

RQ3 - RAG Prompt Template

[INST]

You are an AI assistant that determines whether a GitHub issue is related to a set of retrieved code commit chunks.

Task:

Decide if the commit chunks are relevant to the issue.

Criteria:

Mark RELEVANT = True if:

- Fixes the issue
- Implements the requested feature
- Clearly relates to the same functionality or component

Otherwise, mark RELEVANT = False.

Input:

Issue:

{issue_data}

Commit Chunks:

{commit_chunks}

Output Format (STRICT):

RELEVANT: [True/False]

CONFIDENCE: [0.0-1.0]

Do not include any explanation.

[/INST]